Kubespray · managed Kubernetes · DEVOPS MAY CRY

От виртуалок до Kubernetes и приложения.

Собираем свой кластер, создаём managed-кластер в облаке и разбираем запуск приложения. В начале — краткий итог первой части, устройство стенда и выбранные настройки. Смотреть предыдущий выпуск для понимания этой части необязательно.

5 VM в стенде · 3 ноды Kubernetes

Kubespray v2.31 · containerd + Calico

managed рядом для сравнения

копируемые команды + вывод

краткий вводный раздел: стр. 2–4

подготовка с нуля тоже внутри

Исходники второй части — отдельный релиз на GitHub

Скачать код · video2b-v1.0

t.me/boooostmentor



Виктор Шутов

DevOps-инженер · ментор

boostmentor.ru →

НАЧНИ ЗДЕСЬ · ЧАСТЬ 2 · 1/3

Что запускаем и откуда продолжаем

Во второй части собираем Kubernetes двумя способами: сами через Kubespray и через managed-сервис Yandex Cloud. Затем запускаем наше приложение и сравниваем, что в каждом случае настраиваем мы, а что обслуживает провайдер.

> **i Первую часть смотреть необязательно**
> Ниже — её короткое содержание и принятые решения. Если стенд уже есть, переходи к разделу **07 «Managed-кластер», стр. 34**. Если начинаешь с пустого компьютера, подготовка и создание VM тоже есть в этой тетради: маршрут — на следующем развороте.

Наше приложение — DEVOPS MAY CRY

Это учебная ночная диспетчерская: клиент отправляет заявку на вымышленный инцидент, например «призрак DNS», и получает ответ с назначенным дежурным. Сервер написан на Go. На нём разбираем запуск контейнеров, настройки и доступ к приложению.

ГДЕ ЗАПУСКАЕМ	КАК РАБОТАЕТ ПРИМЕР
Локально, Docker Compose	Два контейнера: API и PostgreSQL. API получает `DB_HOST=postgres` и сохраняет заявки в базе. Этот запуск разбирали в первой части.
В Kubernetes этого выпуска	Запускаем API без PostgreSQL: `DB_HOST` не задан. Приложение формирует ответ и ведёт счётчик в памяти каждой реплики; долговременной истории заявок здесь нет. На этом варианте разбираем кластер и доступ к API.

Код приложения — в `00_DEVOPS_MAY_CRY_APP`. Подробный разбор Compose, HTTP-запросов и выбора хранилища начинается на **стр. 7**.

Что сделали в первой части

1. Разобрали Terraform на небольшом примере: конфигурация, план, применение и state — сохранённые сведения о созданных ресурсах.
2. Разобрали Ansible: список машин, задачи и роли. Затем познакомились с Kubespray — готовыми Ansible-playbook и ролями для установки Kubernetes.
3. Terraform создал сеть и **пять VM**. Из их адресов собрали inventory — список машин и групп для Ansible. **Установка Kubernetes — следующий этап, уже в этой части.**

Код для практики и продолжение серии
Исходники VIDEO2B · релиз video2b-v1.0 на GitHub
Telegram: t.me/boooostmentor · Менторство: boostmentor.ru

Хочешь посмотреть создание стенда на видео — **первая часть по этой ссылке**. Для чтения следующих страниц её просмотр не требуется.

НАЧНИ ЗДЕСЬ · ЧАСТЬ 2 · 2/3

Какие решения по кластеру уже приняли

Пять VM — весь учебный стенд. **В Kubernetes входят три машины.** Их роли заданы в `locals.tf` Terraform-проекта; по этим ролям формируются группы inventory.

МАШИНА	РОЛИ ПОСЛЕ УСТАНОВКИ KUBESPRAY
`node1`	**Control plane** — управление кластером; **etcd** — данные Kubernetes. Через внешний адрес этой VM подключаемся к API с ноутбука.
`node2`	**Control plane + etcd + worker.** Эта машина одновременно управляет кластером и запускает пользовательские приложения.
`node3`	**etcd + worker.** Третий участник хранилища Kubernetes и ещё одна машина для приложений.
`lb1`, `lb2`	Две отдельные Linux-VM для дальнейшей практики внешнего входа. В Kubernetes не входят; наличие VM само по себе ещё не настраивает балансировку.

Что установим на этих машинах

ВЫБОР В ПРИМЕРЕ	ДЛЯ ЧЕГО НУЖЕН
Kubespray v2.31.0 Kubernetes `1.34.7`	Kubespray выполняет установку. Номер Kubernetes — версия самого кластера. Здесь сохранены версии записанного примера; при повторении учитывай срок их поддержки.
containerd	Запускает контейнеры на нодах по запросам kubelet — агента Kubernetes на каждой машине.
Calico	Организует сеть Pod: выделение адресов, связь между Pod и применение сетевых политик.
kube-proxy: iptables	Настраивает обработку обращений к Service — постоянным адресам для групп Pod.
etcd: host	Бинарник etcd запускается как служба Linux под управлением systemd. У нас три участника; для подтверждения записи нужны два.

Адреса разделили по назначению

VM: `10.10.0.0/24` — выбранная для стенда частная подсеть. **Pod:** `10.233.64.0/18`. **Service:** `10.233.0.0/18`. Последние два диапазона оставили из образца конфигурации Kubespray v2.31.0. Эти числа не обязательны для Kubernetes; диапазоны должны подходить по размеру и не пересекаться с сетями, к которым нужен доступ. Подробности — раздел 05, стр. 23.

> ⚠ **Это учебная топология**
>
> Роли совмещены ради компактного стенда. Два control-plane узла не обеспечивают автоматическое переключение нашего внешнего адреса API: подключение с ноутбука настроено на node1. В production отдельно планируют размещение ролей, доступность входа, резервное копирование и восстановление.

НАЧНИ ЗДЕСЬ · ЧАСТЬ 2 · 3/3

Где лежит код и с какого шага начинать

Сначала выбери маршрут по состоянию своего стенда. Для просмотра второй части достаточно предыдущих двух страниц. Для повторения команд нужны облачный каталог, доступ к нему и созданные VM.

СЕЙЧАС У ТЕБЯ	ЧТО ОТКРЫТЬ В ТЕТРАДИ
VM и inventory уже созданы	Продолжай с **managed-кластера, стр. 34**, затем переходи к **preflight и Kubespray, стр. 38**. Не создавай второй экземпляр тех же VM ради нового выпуска.
Старт с нуля	**Раздел 05, стр. 23**: клон Kubespray и его Python-окружение. Затем **подготовка облака, стр. 28**, и **раздел 06, стр. 29**: пять VM и экспорт inventory. Hello-примеры Terraform и Ansible в разделах 03–04 можно пройти для знакомства с инструментами.

Карта каталогов репозитория

В таблице ниже все каталоги, кроме приложения, находятся внутри `01_ЧАСТЬ_1_КЛАСТЕР`. Название каталога сохранено из исходного большого выпуска.

КАТАЛОГ / ВАЖНЫЙ ФАЙЛ	ЧТО ЗДЕСЬ БЕРЁМ
`00_DEVOPS_MAY_CRY_APP`	Приложение, Dockerfile, Compose и манифесты. Его устройство разобрано в начале тетради, на стр. 7.
`03_1.2bis_kubespray_anatomy`	Знакомство с Kubespray. Внутри `kubespray/` — скачанный код установщика и созданное нами `.venv/`.
`04_1.3_five_vm_terraform/terraform`	`locals.tf` — состав VM и роли; `compute.tf` — вызов модуля VM; `inventory.tf` — сборка списка адресов для Ansible.
`05_1.4_managed_cluster/terraform`	Отдельная Terraform-конфигурация: кластер облачного сервиса и группа его рабочих нод. У неё свой state.
`06_1.4bis_node_preflight`	Проверка доступа по SSH, sudo и подготовки Linux перед установкой Kubernetes.
`07_1.5_kubespray_full/inventory/video2`	`inventory.ini` — наши машины и группы; `group_vars/` — версия, сеть, runtime и режим etcd. Скачанный Kubespray берёт настройки этого стенда при запуске.

Если продолжаешь после перерыва

Проверь выбранный облачный каталог и актуальность доступа в терминале запуска. Экспортированные переменные не появляются автоматически в новом окне; IAM-токен может истечь. Свой внешний адрес сверь с разрешённым `/32` в настройках стенда. По предупреждению «inventory не разобран» сначала проверь путь к файлу: это ещё не ошибка SSH или кластера.

Подготовка и проверки описаны в указанных разделах. VM не пересоздаём наугад: при ошибке смотрим конкретную команду и её вывод. Облачные ресурсы оставляем до конца нужной практики; затем удаляем ненужные, чтобы остановить расходы.

00 КАК ЧИТАТЬ ЭТУ ТЕТРАДЬ

Открыл → скопировал → повторил → понял

Инструкция, по которой ты получаешь машины, настраиваешь их и собираешь кластер — и трогаешь каждый слой руками. Команды крупные, широкие, копируемые. У каждого блока метка: где вводить и что должно получиться.

Метки «где вводить»

`LOCAL` твоя машина: terraform, ansible, yc, kubectl

`NODE1` по SSH на ноду стенда

`CLUSTER` kubectl в контексте kubespray / k8s-managed

`POD` внутри client-пода

Структура каждой практической темы

1. **Задача** — что и зачем делаем.
2. **Команды** — широкий терминал, реально копируется, с комментариями.
3. **Ожидаемый вывод** — что должно появиться, и как его читать.
4. **Если сломалось** — чем диагностировать (и отдельный раздел с реальными поломками с записи).
5. **Собес-вывод** — как сформулировать на интервью.

i Заглушки в командах

Везде, где видишь `<FOLDER_ID>`, `<NODE1_IP>`, `<YOUR_IP>/32`, `<CLUSTER_ID>` — подставь своё. Реальные адреса, ID каталога и токены в тетрадь не зашиты намеренно; личные значения живут в `*.tfvars` и `.env`, которые не коммитятся.

⚠ Стенд стоит денег

Облачные VM, диски и managed-кластер расходуют бюджет. Сохраняй нужный стенд между связанными разделами. Когда закончишь всю нужную практику, удали ненужные ресурсы через `terraform destroy` в соответствующем проекте, предварительно проверив план удаления. Ошибки квот разобраны в Troubleshooting.

🤖 СПРОСИ AI ПРАВИЛЬНО — МЕТОД

```
Я собираю Kubernetes-кластер сам: Terraform → Ansible → Kubespray, рядом managed.
Уровень junior→middle. Объясняй коротко, без «магии». После объяснения — НЕ простыня, а:
① один уточняющий вопрос мне на проверку понимания;
② команды, которыми я докажу это на СВОЁМ стенде.
```

↳ проверь: этот шаблон подставляй в начало любого запроса ниже по тетради.

ОГЛАВЛЕНИЕ

Что внутри

Уже создал VM и inventory? Продолжай с раздела 07 на стр. 34. Разделы 01–06 оставлены как справочник и подготовка с нуля. Слева — страница начала раздела.

СТР	РАЗДЕЛ	ЧТО ВНУТРИ
7	**Стенд · DEVOPS MAY CRY**	приложение Go + Postgres · compose · /livez vs /readyz · зачем в кластер без базы
10	**Инструменты · три слоя**	освежение k8s · Terraform / Ansible / Kubespray · словарь и цикл Terraform
13	**Hello Terraform**	файл за файлом · init → plan → apply · state · drift руками
18	**Hello Ansible**	inventory · роль · changed=0 · где идемпотентность заканчивается · precedence · drift
23	**Kubespray и карта установки**	анатомия репо · способы установки · managed vs self · решения · топология 5 VM
29	**Terraform стенда**	карта машин · security group /32 · dynamic-блоки · outputs → inventory.ini
34	**Managed-кластер**	сервисные аккаунты · cluster + node group · outputs · завис в Provisioning
38	**Установка**	preflight · inventory и group_vars · cluster.yml · 7 стадий · PLAY RECAP
44	**Доступ**	admin.conf vs get-credentials · контексты · exec-токен vs сертификат · что установилось
48	**Приложение в кластере**	PSA-метки · selector vs template · topologySpread · rollout · запрос изнутри · managed
55	**Два кластера рядом**	таблица границы ответственности — каждая строка командой в двух контекстах
57	**Troubleshooting**	8 реальных поломок с записи: env, stale plan, квоты, /32, download, LoadBalancer
62	**Собес-тренажёр**	20 вопросов с ловушками по грейдам
67	**Практика**	8 заданий с промтами для ИИ
69	**Командная шпаргалка**	terraform · ansible · kubespray · yc · kubectl с контекстами
72	**AI-запросы**	как просить нейронку по темам + проверка
73	**Итог + что дальше**	что сделал руками · Блок 3: сети, MetalLB, внешний вход, обновление
74	**BoostMentor · что дальше**	Telegram-канал · GitHub · YouTube · менторство

Что за приложение и куда идут данные

DEVOPS MAY CRY — ночная диспетчерская: клиент оставляет заявку на странный инцидент, система назначает дежурного. Все лабораторные блока крутятся вокруг этого приложения. Путь данных: **клиент** (браузер или curl) → **Go API** (принимает HTTP-запрос, проверяет данные, отвечает) → **PostgreSQL** (сюда ложится заявка). Локально поднимаем всё через Docker Compose: два контейнера, база и API, во внутренней сети.

compose.yaml по строкам — `00_DEVOPS_MAY_CRY_APP/compose.yaml`

СТРОКА	ЧТО ДЕЛАЕТ
`services: postgres, api`	Сервис = именованное описание контейнера. Это имена в Compose, не названия Go-файлов.
`DB_HOST: postgres`	Идти к базе по имени сервиса в общей сети Compose. `localhost` внутри контейнера означал бы сам этот контейнер.
`DB_PASSWORD: ${POSTGRES_PASSWORD:?…}`	Подстановка Compose (не Terraform, не Jinja). `:?` — без значения разбор упадёт с подсказкой. `POSTGRES_PASSWORD` — имя для Compose, `DB_PASSWORD` — имя, которое получит Go-процесс.
`depends_on → service_healthy`	api стартует после healthcheck базы (`pg_isready`): база принимает соединения. Это не тест бизнес-запроса.
`ports: "8080:8080"`	Слева порт ноутбука, справа — контейнера. postgres наружу так не публикуем.
`volumes`	`postgres-data` хранит данные отдельно от контейнера; `migrations/001_init.sql` создаёт таблицу на пустой базе. На заполненном томе повторно не проигрывается.
`.env / .env.example`	`.env` Compose подхватывает сам, в git он не попадает. `.env.example` — образец для человека, вместо `.env` не используется.

LOCAL VS Code · cmd/devops-may-cry/main.go — buildStore выбирает хранилище

```go
func buildStore() (store.OrderStore, error) {
  // Без DB_HOST — память процесса. Compose задаёт DB_HOST=postgres,
  // поэтому /readyz докажет связь с PostgreSQL, а /orders — запись строки.
  if os.Getenv("DB_HOST") == "" {
    return store.NewMemory(), nil
  }
  // дальше — подключение к PostgreSQL из DB_HOST/DB_PORT/DB_USER/DB_PASSWORD
}
```

Читаем: связь двух файлов конкретная: compose.yaml передал `DB_HOST=postgres` → main.go прочитал `DB_HOST` → выбран PostgreSQL-store. Манифест Kubernetes первой части `DB_HOST` не задаёт — тот же код выбирает Memory. **Memory** формирует ответ о заявке и крутит счётчик в памяти процесса; историю заявок не хранит даже в коллекцию, у каждой реплики счётчик свой.

СОБЕС «Хранилище выбирается конфигурацией, а не кодом обработчиков: есть `DB_HOST` — PostgreSQL, нет — память процесса. Так я убираю базу из первого теста кластера, не переписывая приложение: тот же образ, те же ручки, другая переменная окружения».

01 СТЕНД · DEVOPS MAY CRY · 2/3

Локальный запуск: Compose, пробы и строка в Postgres

Задача: до всякого Kubernetes увидеть нормальный путь данных: поднять API и базу в Compose, проверить пробы, создать заявку по HTTP и найти её строку прямо в таблице PostgreSQL — мимо приложения. Это контрольная точка: тот же образ и те же ручки позже поедут в кластер.

LOCAL 00_DEVOPS_MAY_CRY_APP — собрать, запустить, доказать, остановить

```
cd 00_DEVOPS_MAY_CRY_APP                    # все Compose-команды отсюда
test -f .env; echo $?                       # 0 = файл есть; пароль не печатаем
docker compose config --services            # yaml разобран: postgres, api
docker compose up -d --build                # образ по Dockerfile + оба сервиса
docker compose ps                           # running + healthy у обоих
docker compose logs --tail=20 api           # какое хранилище выбрал buildStore
curl http://127.0.0.1:8080/livez            # процесс жив; к базе не ходит
curl http://127.0.0.1:8080/readyz           # Ping хранилища: 200 или 503
curl http://127.0.0.1:8080/quote            # обычный пользовательский GET
curl -i -H 'Content-Type: application/json' \
  -d '{"target":"dns-ghost"}' http://127.0.0.1:8080/orders   # -d ⇒ POST
docker compose exec postgres psql -U devopsmaycry -d devopsmaycry \
  -c "SELECT id, target, price_usd, status FROM orders;"      # мимо API
docker compose down                         # без -v: том postgres-data остаётся
```

ОЖИДАЕМЫЙ ВЫВОД

```
0                                  ← test -f .env: файл существует
postgres, api                      ← config --services: две строки
api        running (healthy)       ← compose ps; postgres тоже healthy
{"msg":"devops may cry api started","store":"postgres",...}
{"status":"alive","pod":"..."}     ← /livez
{"status":"ok","pod":"..."}        ← /readyz: соединение с PostgreSQL прошло
HTTP/1.1 201 Created               ← заявка создана, в теле JSON с номером
 id |  target   | price_usd | status
  1 | dns-ghost |      3000 | принят — дежурный выехал
```

Читаем: `store=postgres` в логе — сработала ветка buildStore. `/readyz` → 200 значит Ping к базе прошёл. `201` — handler принял JSON, `store.Create` вставил строку. `SELECT` читает её мимо приложения. Номер заявки и `id` зависят от данных в томе: на повторном запуске строк может быть несколько, id не обязан быть 1.

Если не совпало

`echo $?` → 1: .env нет — готовь его из .env.example вне записи, руками содержимое не печатай. `connection refused` на curl → назад к `compose ps` и `logs api`. `/readyz` → 503 → смотри postgres в `ps` и логи обоих сервисов. Строки в SELECT нет → не идти дальше: сверить 201 и логи postgres. Завершение `up -d` ≠ готовность.

СОБЕС «Локально доказываю полный путь данных: `/readyz` отвечает 200 только после Ping к Postgres, `POST /orders` возвращает 201 с номером, а строку читаю мимо API — `psql` внутри контейнера. `down` без `-v` оставляет том: данные живут отдельно от контейнеров».

01 СТЕНД · DEVOPS MAY CRY · 3/3

/livez, /readyz, ручки API и почему без базы

`/livez` и `/readyz` — обычные HTTP-ручки приложения из `internal/service/server.go`, а не имена Kubernetes. Слева метод и путь, справа обработчик; curl потом ходит по этим точным путям.

РУЧКА	ЧТО ОТВЕЧАЕТ	ЗАЧЕМ
`GET /`	HTML в браузере, JSON-фраза для curl	витрина
`GET /quote`	реплика дежурного + `pod`, который ответил	GET без записи
`GET /status`	имя экземпляра и активный тип store	memory / postgres
`POST /orders`	создать заявку, успех — `201` (`GET /order?on=` — учебный вариант)	настоящая запись
`GET /healthz, /livez`	жив ли HTTP-процесс; к store не ходит	livenessProbe → рестарт
`GET /readyz`	Ping store с таймаутом 1,5 с; нет ответа → `503`	readinessProbe → вон из endpoints
`GET /metrics`	счётчики запросов и принятых заявок	наблюдаемость
`GET /overload, /wound`	CPU и память для лаб; без `LAB_ENDPOINTS_ENABLED=true` → 404	только в изолированном стенде

LOCAL VS Code · internal/service/server.go — handleReady

```
func (s *Server) handleReady(w http.ResponseWriter, r *http.Request) {
  ctx, cancel := context.WithTimeout(r.Context(), 1500*time.Millisecond)
  defer cancel()
  if err := s.store.Ping(ctx); err ≠ nil {      // store выбрал buildStore
    writeError(w, http.StatusServiceUnavailable, "order store not ready")
    return
  }
  writeJSON(w, http.StatusOK, map[string]string{"status": "ok", "pod": s.pod})
}
```

Читаем: один и тот же `/readyz` в Compose проверяет PostgreSQL, а в нашей Kubernetes-лабе — память: что выбрал buildStore, то и пингуется. Liveness к базе не ходит по замыслу: перезапуск процесса внешнюю зависимость не чинит.

i Почему в кластере сначала без базы

Первый запуск в новом кластере проверяет другой слой: может ли собранный кластер запустить контейнер и доставить запрос до приложения. Подключать базу одновременно — ошибка может оказаться и в кластере, и в адресе базы, и в доступах. Цена упрощения: HTTP 200 докажет, что запрос дошёл и обработчик ответил, но не надёжное хранение. Базу в Kubernetes подключить можно — отдельной задачей: где работает, доступ, постоянное хранилище, восстановление.

СОБЕС «`/livez` — “процесс жив?”, к базе не ходит; `/readyz` — “могу обслужить запрос сейчас?”, Ping хранилищу с таймаутом. Ошибка liveness перезапускает контейнер, ошибка readiness временно убирает под из Service endpoints. Поэтому readiness включает базу, а liveness — нет: рестарт здорового процесса базу не вернёт».

02 ИНСТРУМЕНТЫ · ТРИ СЛОЯ · 1/3

Из чего состоит кластер и что делаем в этом блоке

Герои первого видео — снова на доске. В Блоке 1 кластер был готов заранее. Здесь смотрим, как эти компоненты физически появляются на обычных Linux-машинах: сначала машин нет вообще.

КОМПОНЕНТ	КТО НА ДОСКЕ	РОЛЬ
`kube-apiserver`	Зевс, на троне	единственная дверь кластера: все операции с объектами и запросы их состояния. Пользовательский HTTP приложения через него не обязан идти.
`etcd`	Мнемозина, со свитком	хранит API-данные кластера — спецификации и статусы. Не база бизнес-заявок приложения.
`kube-scheduler`	Афина, с совой	решает, на какую ноду поставить новый Pod, и записывает назначение через API. Контейнер kubelet-у не отправляет.
`controller-manager`	Фемида, с весами	сравнивает желаемое состояние с фактическим и доводит кластер до нужного — тоже через API.
`kubelet`	Арес, в доспехах	получает назначение через API и запускает Pod через container runtime. Есть и на control-plane нодах self-managed стенда.
`kube-proxy`	Гермес, вестник	превращает правила Service в сетевые настройки ноды. Запрос к приложению идёт по этому пути, а не через scheduler или API server.

Маршрут VIDEO2A — четыре блока, каждый до наблюдаемого результата

1. **Три инструмента.** Terraform — полный цикл с drift-ом, Ansible — устройство проекта и почему повторный запуск ничего не ломает, Kubespray — что это за Ansible-проект.
2. **Где берут кластер.** Карта способов установки, граница managed / self-managed и решения до установки: топология, версии, runtime, CNI, диапазоны сети.
3. **Два кластера.** Self-managed сами: пять VM Terraform-ом, preflight Ansible-ом, установка Kubespray-ем с живым запуском. Managed — тем же Terraform-языком, control plane прячет провайдер.
4. **Доказать и развернуть.** Кластер настоящий → наше приложение внутри → ответ по HTTP.

> **i Чего в этом блоке нет**
>
> Сети, безопасное обновление и внешний отказоустойчивый вход (два балансировщика) — отдельный маршрут следующего видео. Здесь их не начинаем.

СОБЕС «Control plane принимает решения и хранит состояние, worker-ноды запускают Pod. Scheduler не общается с kubelet напрямую — пишет назначение в API, kubelet своей ноды его забирает. Управление кластером и пользовательский запрос к приложению — два разных пути, проверяю их разными командами».

02 ИНСТРУМЕНТЫ · ТРИ СЛОЯ · 2/3

Terraform, Ansible, Kubespray — три слоя, один список машин

В облаке пусто: ни машин, ни сети, ни кластера. Три инструмента часто считают конкурентами. Возможности частично пересекаются, но в нашем процессе у каждого свой слой, и порядок слоёв физический: машины должны появиться → Linux на них должен быть доступен и настроен → только потом из машин собирается кластер.

ИНСТРУМЕНТ	ЧТО ДЕЛАЕТ	ЧЕГО НЕ ДЕЛАЕТ	АРТЕФАКТ
`Terraform`	создаёт объекты платформы: сеть и пять VM. Обращается к облаку через provider, держит созданное в state	не заходит внутрь Linux, не ставит пакеты и сервисы	state + outputs: адреса созданных VM
`Ansible`	внутри существующих Linux-машин по SSH: установить пакет, положить конфиг, запустить сервис	не создаёт машины в нашем процессе; своей базы объектов облака нет	inventory (список машин + роли) и результат playbook
`Kubespray`	большой готовый Ansible-проект для Kubernetes: по списку машин и ролям раскладывает API server, etcd, kubelet, runtime, CNI. Внутри своих задач вызывает kubeadm	не создаёт VM; отдельный ручной kubeadm-этап после него не нужен	работающий кластер на выбранных нодах

Связь между слоями материальная

1. **Terraform** создаёт VM и отдаёт их адреса (outputs).
2. Эти адреса становятся **списком машин для Ansible**; в том же списке машинам назначаются роли: control plane, etcd, worker.
3. **Kubespray** читает тот же список и те же настройки, когда выполняет установку.

> ⚠ **Что ломается, если слои перепутать**
>
> Машин ещё нет — Ansible некуда подключаться. Список машин неверен — Kubespray настроит не те узлы. kubeadm в колонке Kubespray — инструмент первоначальной настройки и присоединения нод; Kubespray добавляет вокруг него подготовку ОС, runtime, сеть и автоматизацию.

СОБЕС «Terraform и Ansible — разные слои. Terraform управляет существованием объектов у провайдера — сети, машины — и держит их в state, поэтому умеет plan, сверку и destroy. Ansible работает внутри уже существующих машин по SSH. Kubespray — Ansible-проект, который из этих машин собирает Kubernetes. Пограничные случаи есть с обеих сторон, но границу держу: существование — Terraform, наполнение — Ansible».

02 ИНСТРУМЕНТЫ · ТРИ СЛОЯ · 3/3

Словарь Terraform и цикл init → validate → plan → apply

Инфраструктура кликами в консоли плохо воспроизводится. Terraform решает это декларативно: описываешь итоговое состояние, он сравнивает код, state и внешнюю систему и строит план.

ТЕРМИН	ЧТО ЭТО
`provider`	плагин для конкретной системы (Yandex Cloud, AWS, Kubernetes, VMware, локальные файлы): переводит блоки конфигурации в вызовы API
`resource`	один объект под управлением Terraform: VM, сеть, файл. В блоке нет слова «создай» — только что должно существовать
`variable`	вход root module: тот же код с другой zone или подсетью без правки ресурсов. Значения — из tfvars
`local`	именованное выражение внутри конфигурации: вычислить раз, звать по имени. Через `-var-file` не задаётся
`output`	то, что root module отдаёт наружу: человеку или следующему инструменту. Позже так отдадим Ansible адреса VM
`state`	память Terraform: адрес в коде ↔ реальный объект и его последние атрибуты. Руками не удаляют: объект останется жить без хозяина
`drift`	управляемый объект изменили вне Terraform (консоль, руками на сервере): реальность уехала от кода, state ещё не знает. Ловится plan-ом

LOCAL рабочий цикл — одинаков для локальных файлов и для облака

```
terraform init       # required_providers → качает providers, пишет lock
terraform validate   # синтаксис и связность кода; никуда не подключается
terraform plan       # код + state + реальность → add / change / destroy
terraform apply      # тот же план, явное yes → реальность к коду, в state
terraform plan       # контрольная точка: никто не менял → No changes
```

Читаем: `init` ресурсы не трогает — это установка инструмента. Ниже словарь в живом синтаксисе; apply на нём не делаем.

LOCAL VS Code · 01_1.1_hello_terraform/reference/yandex-cloud/main.tf

```
provider "yandex" { zone = var.zone }          # credentials в коде нет
resource "yandex_compute_instance" "node1" {   # одна VM, без слова «создай»
  name = "node1"
  resources {                                  # 2 ядра, 4 ГБ — описание
    cores  = 2
    memory = 4
  }
  # ещё platform_id, zone = var.zone, boot_disk из var.image_id, nat = true
}
```

СОБЕС «State — память Terraform, не бэкап: без него plan не отличит “создать” от “уже существует”, а его удаление объекты не удаляет. Цикл: init → validate → plan → apply → повторный plan, который обязан ответить “No changes”».

03 HELLO TERRAFORM · 1/5

Root module: versions, providers, variables, tfvars

Задача: прочитать маленький root module целиком до первого запуска. Лаборатория создаст три файла в `build/`: dispatcher.env, journal.txt, routing-summary.json — на них виден весь цикл без облака и без счёта. Root module — каталог с .tf, откуда запускаем terraform: все .tf читаются как одна конфигурация, подпапки сами не подключаются.

LOCAL 01_ЧАСТЬ_1_КЛАСТЕР/01_1.1_hello_terraform/versions.tf + providers.tf

```
terraform {
  required_version = "≥ 1.9.0, < 2.0.0"   # версия самой программы terraform
  required_providers {
    local = {                              # короткое имя провайдера здесь
      source  = "hashicorp/local"          # откуда плагин
      version = "2.5.2"                    # закреплено → .terraform.lock.hcl
    }
  }
}
# providers.tf — параметры экземпляра; local-у не нужны token и region
provider "local" {}
```

Читаем: `required_providers` говорит, какой плагин установить; `provider` задаёт параметры его работы. Ни один из этих файлов журнал смены ещё не описал.

LOCAL variables.tf (объявление входа) + env/demo.tfvars (значения запуска)

```
variable "dispatcher" {
  description = "Имя дежурного, которое попадёт в dispatcher.env"
  type        = string                     # требует строку
  validation {                             # ошибку ловим до создания файлов
    condition     = length(trimspace(var.dispatcher)) ≥ 2
    error_message = "Имя дежурного должно содержать минимум два символа."
  }
}
# env/demo.tfvars — ресурсы не создаёт, только передаёт значения variables.tf
office_name = "DEVOPS MAY CRY"
dispatcher  = "Виктор"                     # → var.dispatcher
shift       = "22:00-06:00"
routes = {                                 # map(number): инцидент → цена
  dns-ghost   = 3000
  disk-full   = 5000
  memory-leak = 7000
}
```

Читаем: объявление входа и его значение — разные вещи. Файл лежит в `env/` под своим именем, поэтому передаём явно: `-var-file=env/demo.tfvars`. Второй tfvars с тем же state меняет те же ресурсы — отдельного окружения не даёт.

СОБЕС «Provider — плагин, который переводит resource-блоки в вызовы API конкретной системы. `required_providers` выбирает плагин и версию, `init` скачивает и фиксирует её в lock-файле, `provider`-блок конфигурирует экземпляр: регион, endpoint, credentials. Credentials в код не кладу — окружение или штатный механизм провайдера».

locals, шаблон, resource: одно значение до журнала

Задача: проследить одно значение целиком: `demo.tfvars` задаёт «Виктор» → Terraform получает `var.dispatcher` → `templatefile` передаёт `dispatcher` → `${dispatcher}` в шаблоне становится «Виктор» → `local_file` пишет готовый текст в `build/journal.txt`.

LOCAL locals.tf — внутренние вычисления, снаружи не задаются

```
locals {
  output_dir = "${path.module}/build"       # path.module = каталог root
  routing_summary = {                       # один объект для JSON-шаблона
    office      = var.office_name
    dispatcher  = var.dispatcher
    shift       = var.shift
    route_count = length(var.routes)        # три маршрута → 3
    routes      = var.routes
  }
}
```

LOCAL templates/journal.tftpl → resources.tf (resource "local_file" "journal")

```
# journal.tftpl — текстовая заготовка; Terraform её отдельно не исполняет
Дежурный: ${dispatcher}
%{ for incident, price in routes ~}         # цикл по map; ~ режет пробелы
${incident}: ${price}                       # dns-ghost → «dns-ghost: 3000»
%{ endfor ~}

# resources.tf — кто читает шаблон и что ему передаёт
resource "local_file" "journal" {           # в state: local_file.journal
  filename        = "${local.output_dir}/journal.txt"
  file_permission = "0644"
  content = templatefile("${path.module}/templates/journal.tftpl", {
    dispatcher = var.dispatcher             # имя в шаблоне = значение входа
    routes     = var.routes
  })
}
```

Читаем: `local.output_dir` в `filename` — ссылка на значение из locals.tf. Шаблон не ищет demo.tfvars сам: имена ему передаёт только второй аргумент `templatefile`. Тот же приём, что у Jinja в Ansible — данные отдельно, форма отдельно, — но синтаксис другой.

i Имя ресурса ≠ имя файла

`local_file` — тип ресурса провайдера local, `journal` — имя внутри Terraform; вместе — адрес `local_file.journal` в `state list`. Путь на диске задаёт `filename`. Drift позже создадим в выходном `build/journal.txt`; правка `journal.tftpl` — уже изменение кода желаемого состояния.

СОБЕС «Variable — вход, значение приходит из tfvars через `-var-file`; local — именованное выражение внутри конфигурации, снаружи не задаётся; `templatefile` получает явный набор параметров — шаблон сам tfvars не читает. Цепочку “tfvars → var → шаблон → файл” показываю на одной строке журнала».

resources.tf целиком, depends_on и outputs.tf

Задача: дочитать оставшиеся два ресурса и outputs. `dispatcher_env` — первый ресурс, с него в видео началось чтение; `routing_summary` — JSON-сводка с явной зависимостью.

LOCAL resources.tf — dispatcher_env и routing_summary

```
resource "local_file" "dispatcher_env" {    # адрес: local_file.dispatcher_env
  filename        = "${local.output_dir}/dispatcher.env"
  file_permission = "0644"
  content = templatefile("${path.module}/templates/dispatcher.tftpl", {
    office     = var.office_name            # слева имя в шаблоне, справа вход
    dispatcher = var.dispatcher
    shift      = var.shift
  })
}

resource "local_file" "routing_summary" {
  filename        = "${local.output_dir}/routing-summary.json"
  file_permission = "0644"
  content = templatefile("${path.module}/templates/summary.tftpl", {
    summary_json = jsonencode(local.routing_summary)   # объект → строка JSON
  })
  depends_on = [                           # учебная явная зависимость
    local_file.dispatcher_env, local_file.journal,
  ]
}
```

LOCAL outputs.tf — что root module отдаёт наружу

```
output "artifacts" {
  description = "Публичный контракт root module для следующего шага"
  value = {
    dispatcher_env  = local_file.dispatcher_env.filename   # атрибут ресурса
    journal         = local_file.journal.filename
    routing_summary = local_file.routing_summary.filename
  }
}
```

Читаем: output не копирует файл и не хранит его версию — сообщает результат. Маршрут перед запуском: versions + provider готовят инструмент → variables + tfvars дают входы → locals вычисляют → templatefile собирает текст → resources управляют тремя файлами → outputs возвращают их пути. Выполнение увидим только после plan и apply.

⚠ Terraform не исполняет .tf по очереди

Он собирает одну конфигурацию и строит порядок операций из зависимостей между ресурсами. Обычные ссылки на атрибуты (`local_file.journal.filename`) уже дают граф автоматически; `depends_on` здесь — учебный пример явной зависимости, когда ссылки атрибутом нет.

СОБЕС «Порядок операций Terraform строит из графа ссылок между ресурсами, а не из имён файлов. `depends_on` нужен только когда зависимость не выражена атрибутом. Output — публичный контракт root module: дальше по такому же output Ansible получит адреса созданных VM».

Первый цикл и три доказательства после apply

Задача: прожить цикл руками от init до «No changes». Каждая команда отвечает на свой вопрос: инструмент готов? код согласован? что Terraform собирается сделать? что сделал? — и три отдельных доказательства после apply.

LOCAL 01_ЧАСТЬ_1_КЛАСТЕР/01_1.1_hello_terraform — цикл

```
cd ../01_ЧАСТЬ_1_КЛАСТЕР/01_1.1_hello_terraform   # root module лабы
terraform init                                   # provider local 2.5.2 + lock
terraform validate                               # код: синтаксис, ссылки, типы
terraform plan -var-file=env/demo.tfvars         # прогноз по значениям из файла
terraform apply -var-file=env/demo.tfvars        # тот же план + yes после чтения
```

ОЖИДАЕМЫЙ ВЫВОД

```
Terraform has been successfully initialized!
Success! The configuration is valid.
Plan: 3 to add, 0 to change, 0 to destroy.       ← чистый первый запуск
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Читаем: после init ни один ресурс не создан — установлен плагин. validate не смотрит ни в state, ни на диск. plan — прогноз, на диске ещё пусто. Если план на подтверждении отличается от прочитанного — **no** и разбираться.

LOCAL три доказательства: диск, память Terraform, контрольный plan

```
cat build/journal.txt                            # 1) физический результат
terraform state list                             # 2) что Terraform считает своим
terraform plan -var-file=env/demo.tfvars         # 3) код + state + диск → ноль
```

ОЖИДАЕМЫЙ ВЫВОД

```
Дежурный: Виктор
disk-full: 5000                        ← map обходится по алфавиту ключей
dns-ghost: 3000
memory-leak: 7000
local_file.dispatcher_env
local_file.journal
local_file.routing_summary
No changes. Your infrastructure matches the configuration.
```

Читаем: у каждого файла на диске есть управляемый двойник в state — ровно три адреса из resources.tf. Повторный plan без правок обязан быть пустым; непустой diff не прятать — искать ручную правку или изменение входов.

i Где лежит state

`terraform.tfstate` рядом с .tf-файлами — JSON состояния, не код и не бэкап содержимого систем. Здесь секретов нет; в облачном state они бывают, поэтому целиком на экран его не печатают. Для команды — remote backend с блокировкой, чтобы два apply не перетёрли друг друга.

СОБЕС «`init` готовит каталог: backend, providers, modules, `.terraform` и lock-файл — инфраструктуру не создаёт. `plan` сравнивает код, state и реальность; `apply` строит тот же план и после явного yes приводит реальность к коду. Контрольная точка цикла — повторный plan с “No changes”».

Drift: ломаем файл руками, plan ловит, apply возвращает

Задача: развести три вещи, которые легко перепутать: **конфигурация** — что хотим (tf-файлы), **state** — что Terraform помнит о созданном, **реальный объект** — что существует на диске или в облаке. Пока все трое совпадают, plan отвечает «No changes». Drift — не любой diff: правка кода — нормальный workflow. Drift — когда управляемый объект изменили **вне** Terraform. Код не трогаем, ломаем реальность.

LOCAL тот же каталог, state после apply — создать drift, поймать, вернуть, убрать

```
printf '\nmanual_change=true\n' >> build/journal.txt   # правка мимо Terraform
cat build/journal.txt                                  # последняя строка не из шаблона
terraform plan -var-file=env/demo.tfvars               # диск vs state vs код
terraform apply -var-file=env/demo.tfvars              # правка — мусор: возвращаем код
cat build/journal.txt                                  # ручной строки больше нет
terraform plan -var-file=env/demo.tfvars               # контрольный план
terraform destroy -var-file=env/demo.tfvars # уборка стенда, не лечение drift
```

ОЖИДАЕМЫЙ ВЫВОД

```
Дежурный: Виктор … memory-leak: 7000
manual_change=true                        ← cat после printf
# local_file.journal will be created
Plan: 1 to add, 0 to change, 0 to destroy.     ← drift пойман сверкой
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Дежурный: Виктор … memory-leak: 7000     ← cat: manual_change нет
No changes. Your infrastructure matches the configuration.
Destroy complete! Resources: 3 destroyed.
```

Читаем: о правке Terraform узнаёт только при следующем plan или apply. После apply реальность снова совпадает с кодом — и это сделано через инструмент, а не второй ручной правкой. После destroy state пуст, `build/` пуст — лабораторию можно повторить с чистого листа.

«will be created» ≠ файл удалён

cat только что показал файл на месте вместе с ручной строкой. Provider local сверяет содержимое по контрольной сумме: после дописывания это уже не тот управляемый экземпляр, чей хеш лежит в state, поэтому Terraform 1.13.4 и local 2.5.2 сразу предлагают восстановление — «1 to add», без плашки Objects have changed outside of Terraform. У облачных provider тот же drift чаще выглядит как update in-place или замена. Форма разная, суть одна.

⚠ Развилка — её спрашивают на собесе

Правка **ошибочна** → прочитанный, осознанный apply возвращает описанное состояние. Правка **нужна** → сначала переносишь её в код, шаблон или переменные, потом смотришь новый plan и добиваешься пустого. Чего не делаем никогда: не удаляем state, чтобы «сбросить» проблему, и не запускаем apply вслепую — за ручной правкой в проде может стоять чей-то срочный хотфикс.

СОБЕС «Drift — расхождение реального объекта с конфигурацией, возникшее вне Terraform-workflow. Ловлю штатной сверкой — plan на чистом коде. Дальше развилка: правка ошибочна — прочитанный apply возвращает описанное состояние; правка нужна — переношу в код и добиваюсь пустого плана. State не удаляю, apply вслепую не запускаю. В зрелых командах plan гоняют по расписанию в CI, а ручные правки закрыты правами».

Ansible: что сделать, где выполнить и как повторить

Машина уже есть. На ней нужно создать каталог и положить конфиг. На одной машине — руками. Когда машин несколько, описываешь действия один раз и повторяешь без ручного обхода. Тренируемся на ноутбуке: собираем `dispatcher.conf` диспетчерской DEVOPS MAY CRY. Go-приложение пока не трогаем.

СЛОВО	ЧТО ЭТО	У НАС
`inventory`	список машин и их групп — «где выполнять»	`inventory/hosts.ini`
`host / group`	одна машина / несколько под общим именем; host может быть в нескольких группах	`localhost` в группе `night_office`
`task / module`	шаг работы / программа, которая его выполняет. Модуль описывает состояние: не `mkdir`, а «каталог существует»	`file`, `template`, `slurp`, `debug`
`play / playbook`	play связывает группу машин с набором задач; playbook — YAML с одним или несколькими plays	`playbooks/site.yml` — один play
`role`	каталог стандартной структуры: tasks, defaults, templates, handlers — чтобы переиспользовать	`roles/dispatcher`
`handler`	задача, которую вызывают через `notify` — только когда что-то реально изменилось	«Report a configuration change»
`become`	выполнить от другого пользователя (обычно root через sudo)	`become = false` — работаем в `/tmp`

LOCAL 01_ЧАСТЬ_1_КЛАСТЕР/02_1.2_hello_ansible — порядок чтения из README

```
02_1.2_hello_ansible/
├── ansible.cfg                      # 1  где Ansible ищет inventory и roles
├── inventory/
│   ├── hosts.ini                    # 2  группа night_office + localhost
│   ├── group_vars/all.yml           # 3  общие значения
│   ├── group_vars/night_office.yml  # 4  значения группы
│   ├── host_vars/localhost.yml      # 5  значение конкретного host
│   └── dynamic_cloud_example.yml    # справочник: динамический inventory
├── roles/dispatcher/
│   ├── defaults/main.yml            # 6  самые слабые defaults роли
│   ├── tasks/main.yml               # 7  действия по порядку
│   ├── templates/dispatcher.conf.j2 # 8  шаблон результата
│   └── handlers/main.yml            # 9  реакция только на изменение
├── playbooks/site.yml               # 10 play: группа night_office + role
├── playbooks/precedence.yml         # отдельный опыт: gear из play vars
└── non_idempotent_examples.yml      # справочник, к site.yml не подключён
```

СОБЕС «Inventory говорит, где выполнять; play связывает группу машин с задачами; задача вызывает модуль, а модуль описывает состояние, а не команду. Role — переиспользуемый каталог с задачами, шаблонами и handlers. Ansible ходит по SSH и не требует агента на машинах; у нас `ansible_connection=local` — та же модель на ноутбуке».

04 HELLO ANSIBLE · FILE WALK · 2/5

site.yml → role dispatcher: откуда берётся каждое значение

Задача: пройти проект по ссылкам: `ansible.cfg` — где искать входы, `hosts.ini` — машины, `site.yml` — какая им работа, роль — как переменные становятся файлом. До запуска знаешь, что произойдёт и почему.

ФАЙЛ	ЧТО ВНУТРИ И ЧТО ЗАПОМНИТЬ
`ansible.cfg`	`inventory = inventory/hosts.ini`, `roles_path = roles`, `become = false` — настройки Ansible
`inventory/hosts.ini`	`[night_office]` — группа; `localhost ansible_connection=local` — без SSH
`group_vars/all.yml`	для всех hosts: `office_path`, `dispatcher_name`, `gear` «Фонарь…»
`group_vars/night_office.yml`	только группа: «дежурный ночной смены», «Рация…», словарь `call_prices`
`host_vars/localhost.yml`	одна машина: только `gear` «Ключ-карта…»; остальное — из своих источников
`playbooks/site.yml`	один play: `hosts: night_office` — где; `role: dispatcher` — откуда задачи
`dispatcher/defaults/main.yml`	самые слабые значения: `office_path: /tmp/devops-may-cry`, `call_prices: {}`
`dispatcher/tasks/main.yml`	4 задачи: `file` каталог → `template` + `notify` → `slurp` → `debug`; tags config/verify
`templates/dispatcher.conf.j2`	Jinja2: `gear={{ gear }}`; цикл по `call_prices | dictsort` → `price_dns_ghost=3000`
`dispatcher/handlers/main.yml`	имя совпадает с `notify`; здесь только `debug`, в service-role был бы reload

LOCAL проверки до запуска — ничего не меняют

```
cd ../02_1.2_hello_ansible
ansible --version                          # config file → наш ansible.cfg?
ansible-inventory --graph                  # дерево групп, без подключения
ansible-playbook playbooks/site.yml --syntax-check  # YAML + role
```

ОЖИДАЕМЫЙ ВЫВОД

```
  config file = .../02_1.2_hello_ansible/ansible.cfg
@all:
  |--@night_office:
  |  |--localhost
playbook: playbooks/site.yml
```

Читаем: `--graph` рисует не сеть, а группы inventory: `night_office` существует. `--syntax-check` загрузил playbook и роль, задач не выполнял. Что модули отработают — докажет только запуск.

СОБЕС «Playbook — точка входа: выбирает hosts и подключает roles. Role — переиспользуемая единица: tasks, defaults, templates, handlers. Переменные Ansible собирает для host по приоритету: defaults роли → group_vars/all → group_vars группы → host_vars».

Первый run, PLAY RECAP и честный changed=0

Задача: запустить `site.yml`, прочитать RECAP, затем поверить не отчёту, а файлу. Второй запуск без изменений должен дать `changed=0` и молчащий handler. Если пример уже запускали, первый run тоже даст ноль — это не ошибка, рабочие данные ради красивой цифры не удаляем.

LOCAL первый запуск — каталог и файл создаются

```
ansible-playbook playbooks/site.yml
```

ОЖИДАЕМЫЙ ВЫВОД ПРИ ЧИСТОМ ПЕРВОМ ЗАПУСКЕ

```
TASK [dispatcher : Ensure the office directory exists] ****
changed: [localhost]
TASK [dispatcher : Render the shift configuration] ****
changed: [localhost]
RUNNING HANDLER [dispatcher : Report a configuration change] ****
ok: [localhost] ⇒ { "msg": "dispatcher.conf changed; a service role..." }
PLAY RECAP ****
localhost : ok=5  changed=2  unreachable=0  failed=0  skipped=0
```

Читаем: `changed=2` — каталог и template. Handler появился потому, что template реально изменил файл и `notify` поставил его в очередь. RECAP считает задачи, а не содержимое: что в файле правильные значения — он не доказывает.

LOCAL физический результат и повторный запуск

```
cat /tmp/devops-may-cry/dispatcher.conf     # какой уровень переменных победил
ansible-playbook playbooks/site.yml         # ничего не менял — жду changed=0
```

ОЖИДАЕМЫЙ ВЫВОД

```
# Файл управляется Ansible: ручная правка будет возвращена к описанному виду.
office=/tmp/devops-may-cry
dispatcher=дежурный ночной смены          # group_vars/night_office > all
gear=Ключ-карта из host_vars              # host_vars > group_vars > defaults
price_disk_full=5000                      # 3 строки из call_prices, dictsort
price_dns_ghost=3000
price_memory_leak=7000

localhost : ok=4  changed=0  failed=0     # RUNNING HANDLER в выводе нет
```

Читаем: подписи в значениях показывают победителя на каждом уровне. Второй run: каждый модуль сравнил желаемое с текущим и не тронул то, что уже правильно. `changed=0` доказывает только это — не «playbook идемпотентен всегда и на любой машине».

СОБЕС «Зелёный RECAP — не приёмка. Я читаю созданный файл и сверяю значения, потом гоняю playbook второй раз: `changed=0` и handler молчит. Если changed повторяется — ищу задачу, которая всегда отчитывается changed, и чиню её параметры, а не игнорирую отчёт».

За счёт чего changed=0 — и где это заканчивается

Задача: развести задачу и модуль. Задача — запись с name и параметрами; модуль — программа, которая её выполняет. Сравнение состояния живёт внутри модуля: он сначала читает текущее и меняет систему только при расхождении. Файл-справочник `non_idempotent_examples.yml` к `site.yml` не подключён — читаем, не запускаем.

МОДУЛЬ	КАК УЗНАЁТ СОСТОЯНИЕ	ПОВТОРНЫЙ ПРОГОН
`apt state: present`	спрашивает dpkg, стоит ли пакет	стоит → **ok**, система не тронута
`file state: directory`	существование, тип, права	совпало → **ok** (видели на 2-м run)
`template`	рендерит и сравнивает с файлом на диске	**changed** только при реальной разнице → notify
`service state: started`	«сервис должен работать» — требование	работает → **ok**
`service state: restarted`	действие, не состояние	перезапуск КАЖДЫЙ прогон → место в handler
`shell: echo … >> file`	не знает желаемого состояния	всегда **changed**, файл растёт бесконечно
`command` / `raw`	та же граница: произвольная команда	всегда **changed**, пока автор не добавит логику

✓ **Три исправления для «плохого shell»**

1) специализированный модуль: `lineinfile: line: "debug=true"` — строка должна присутствовать (дубли не чистит, без файла нужен `create: true`). **2)** `command … args: creates: /opt/app/.installed` — маркер есть → команда пропускается целиком; `removes` — зеркально. **3)** честный `changed_when: "'updated' in sync_result.stdout"` через `register` — только если по выводу правда видно изменение. `changed_when: false` «для красоты» — это выключенная сигнализация.

LOCAL `precedence переменной gear — от слабого к сильному`

```
roles/dispatcher/defaults/main.yml  gear: "Фонарь из defaults роли" # слабый
inventory/group_vars/all.yml        gear: "Фонарь из group_vars/all"
inventory/group_vars/night_office.yml gear: "Рация из group_vars/night_office"
inventory/host_vars/localhost.yml   gear: "Ключ-карта из host_vars" # site.yml
playbooks/precedence.yml  vars:     gear: "Компас из play vars" # сильнее всех

ansible-playbook playbooks/precedence.yml   # отдельный опыт: debug → Компас…
```

Читаем: host_vars перекрыл одну переменную, остальные остались из своих источников. Это цепочка именно этой лаборатории, а не полная таблица приоритетов Ansible. Секреты так не показывают: Vault или secret manager, и не печатать через `debug`.

СОБЕС «Идемпотентность — свойство выбранного модуля и его параметров, а не Ansible в целом: `service started` идемпотентен, `restarted` перезапускает каждый прогон, `shell` вообще не знает желаемого состояния. Сказать „Ansible гарантирует идемпотентность любого playbook“ — ошибка; „не поддерживает вообще“ — тоже».

Drift: ломаем dispatcher.conf руками, предсказываем, возвращаем

Задача: первая строка шаблона обещает: «ручная правка будет возвращена». Проверяем. Сценарий из жизни: ночью инцидент, кто-то дописал строку прямо в прод-конфиг, «потом внесу в Ansible». Меняем результат в `/tmp`, не шаблон — фактическое состояние разошлось с описанием.

LOCAL drift → предсказание → restore → файл → контроль

```
printf '\nmanual_override=true\n' \
  >> /tmp/devops-may-cry/dispatcher.conf          # правка в обход Ansible
ansible-playbook playbooks/site.yml --check --diff  # репетиция без правок
ansible-playbook playbooks/site.yml --diff          # restore по-настоящему
cat /tmp/devops-may-cry/dispatcher.conf            # верю файлу, не отчёту
ansible-playbook playbooks/site.yml                # контроль: changed=0
```

ОЖИДАЕМЫЙ ВЫВОД

```
--- before: /tmp/devops-may-cry/dispatcher.conf
+++ after: .../roles/dispatcher/templates/dispatcher.conf.j2
 price_memory_leak=7000
-
-manual_override=true
changed: [localhost]
localhost : ok=5  changed=1  failed=0     # --check: предсказано, файл цел
RUNNING HANDLER [dispatcher : Report a configuration change] ****
localhost : ok=5  changed=1  failed=0     # restore: тот же diff, переписан
localhost : ok=4  changed=0  failed=0     # контроль: RUNNING HANDLER нет
```

Читаем: агента, который следит за файлом, нет — drift виден только при прогоне. `--check` сравнил рендер шаблона с диском и предсказал удаление одной строки, ничего не меняя. Restore применил байт в байт тот же diff, реальное изменение снова подняло handler. После `cat` строки `manual_override` нет, файл заканчивается тремя `price_*`. Контроль — ноль правок, handler молчит.

⚠ Границы репетиции

Поддержка `--check` и `--diff` зависит от модуля: `template` и `file` умеют сравнивать, а `command` в check-режиме по умолчанию пропускается. Зелёная репетиция — сильный сигнал, не гарантия реального прогона. Handler в check-выводе может появиться (наш `debug` работает в check) — файл при этом не исправлен. В production check-прогон гоняют по расписанию: предсказанный changed на машине, которую никто не должен был трогать, — сигнал тревоги.

СОБЕС «Handler — задача, которую task ставит в очередь через `notify`, и только когда сообщила changed. В конце play очередь дедуплицируется: пять изменённых конфигов с notify на один handler дают один restart, не пять; ни один не изменился — рестарта нет. У нас handler сработал на первом run и на restore после drift — и молчал на обоих повторах».

05 KUBESPRAY · АНАТОМИЯ · 1/5

Kubespray: большой Ansible-проект, закреплённый tag v2.31.0

Задача: Kubespray — проект Kubernetes SIGs; тот же словарь, что в маленькой лаборатории, только результат — установленный кластер. Клонируем не `main` (меняется каждый день), а release-tag, готовим ему отдельную .venv. `cluster.yml` пока не запускаем.

LOCAL 03_1.2bis_kubespray_anatomy — клон, tag, .venv (клон уже есть → clone пропусти)

```
git clone --branch v2.31.0 --depth 1 \
  https://github.com/kubernetes-sigs/kubespray.git
cd kubespray
git remote get-url origin                    # откуда клон
git describe --tags --exact-match            # на чём стоит HEAD, а не имя папки
git status --short                           # нет ли правок playbooks и roles
python3 -m venv .venv                        # зависимости только этого клона
.venv/bin/pip install -r requirements.txt    # pip читает список из файла
.venv/bin/pip check
.venv/bin/ansible-playbook --version         # именно программа из .venv
export KUBESPRAY_DIR="$PWD"                  # путь к клону для других лаб
```

ОЖИДАЕМЫЙ ВЫВОД

```
https://github.com/kubernetes-sigs/kubespray.git
v2.31.0                                  # HEAD ровно на tag
No broken requirements found.
ansible-playbook [core 2.18.x]           # executable = .../.venv/bin/...
```

Читаем: до clone — `git --version`, `python3 --version` (нужен 3.11–3.13). v2.31.0 требует ansible-core ≥ 2.18.0 и < 2.19.0; пакет `ansible 11.x` из requirements — не номер core. Другой origin, tag или core 2.19 — стоп до установки.

LOCAL что читать в клоне — порядок из README лабораторной

```
kubespray/  (tag v2.31.0)
├── cluster.yml            # обёртка: import_playbook: playbooks/cluster.yml
├── upgrade-cluster.yml    # обновление: дальнейшая практика
├── inventory/sample/      # официальный образец inventory и group_vars
├── roles/kubespray_defaults/   # defaults и проверки версий (checksums.yml)
└── roles/container-engine/ etcd/ kubernetes/ network_plugin/   # этапы сборки
```

i Две папки с похожими именами

`03_1.2bis_kubespray_anatomy` — клон установщика (playbooks, roles, .venv). `07_1.5_kubespray_full` — наши inventory и group_vars, не второй Kubespray. Клон один; после подготовки VM копируем в него входные файлы и запускаем `cluster.yml`. Матрица release подтверждает 1.34.7 сейчас и 1.35.4 для upgrade.

СОБЕС «Kubespray — большой Ansible-проект: inventory говорит, какие машины станут control plane, etcd и worker; group_vars — какую версию, runtime, CNI и CIDR ставить; `cluster.yml` связывает это с готовыми roles. Версию закрепляю tag-ом и проверяю `git describe --exact-match`, не именем папки».

Карта способов получить Kubernetes: задача × кто владеет результатом

Способов больше десятка, и все честные — они решают разные задачи. Вместо спора «что лучше» две оси: какую задачу решаем и кто владеет результатом. В этом выпуске нужны оба стенда: self-managed через Kubespray, чтобы увидеть кухню, и managed через Terraform, чтобы сравнивать видимые ноды и обязанности, а не рекламу.

СПОСОБ	КТО ДЕЛАЕТ CONTROL PLANE	КОГДА БРАТЬ
`Minikube / kind / k3d`	инструмент, на твоём ноутбуке — ноды как контейнеры	учить объекты и API за минуты; моделировать несколько нод и сбои. Общая машина = общая точка отказа
`kubeadm`	ты: базовая инициализация Kubernetes и присоединение нод	когда нужен минимальный слой; ОС, runtime и сеть готовишь сам
`Kubespray`	ты, но через готовые Ansible-roles; внутри вызывает kubeadm	свои Linux-машины и повторяемая установка; control plane, etcd и обновления — твои
`Managed (облако)`	провайдер: control plane и etcd скрыты за облачным API	облако подходит и хочешь передать обслуживание control plane по SLA

i kubeadm внутри Kubespray

kubeadm выполняет первоначальную настройку Kubernetes и присоединение нод. Kubespray вызывает его внутри своих задач и добавляет вокруг подготовку ОС, runtime, сети и остальную автоматизацию. После Kubespray отдельный этап kubeadm руками не запускаем.

✓ Вопрос с доски: managed, Kubespray или kubeadm?

Облако подходит и хотим отдать control plane провайдеру — managed. Обслуживаем свои машины и нужна повторяемая установка — Kubespray. kubeadm — базовая инициализация, которую Kubespray организует через Ansible. Выбор зависит от задачи и ответственности команды, а не от одного «лучшего» инструмента.

⚠ Почему для разбора эксплуатации взяты отдельные VM

Несколько контейнерных нод на одном ноутбуке не эквивалентны независимым Linux-машинам. Учебные локальные кластеры хороши для API и объектов, но машины, роли и файлы установки видно только на настоящих нодах.

СОБЕС «Раскладываю способы по двум осям: задача и кто владеет результатом. Локальные — учить API. Self-managed (kubeadm, Kubespray) — control plane, etcd и обновления мои. Managed — control plane у провайдера. Kubespray я беру для повторяемой установки на свои машины; внутри он всё равно зовёт kubeadm».

Managed и self-managed — где проходит граница

Задача: слова managed и self-managed остаются маркетингом, пока граница не проведена по слоям. Проводим её по выбранному здесь managed-сервису; детали worker-обслуживания, доступных флагов и SLA у других облаков могут отличаться.

СЛОЙ	SELF-MANAGED (KUBESPRAY)	MANAGED (ОБЛАКО)
Linux-машины, диски, сеть	провайдер даёт базу — VM, диски, сеть; обслуживаем мы	control-plane узлов в списке машин нет
Control plane: api-server, scheduler, controller-manager	наши, на node1/node2; версии и флаги — наши	скрыт за облачным API, зайти нельзя; версии и флаги меняются в темпе провайдера, по SLA
etcd	наш, три участника на node1–node3	скрыт вместе с control plane
Сертификаты, обновления control plane	мы: upgrade-cluster.yml, snapshot etcd	провайдер
Worker-ноды	мы полностью	параметры группы задаём мы; обслуживание частично сервис (в Terraform будет `auto_repair`)
Приложения, данные, права, стоимость	ниже границы в обоих случаях — наша ответственность	

Твоя зона ответственности — managed её не забирает

- **Приложения и манифесты** — что запускаем · **requests, limits, rollout, probes** — с какими ресурсами и проверками готовности
- **Ingress, RBAC, NetworkPolicy** — как разрешаем доступ · **данные, бэкапы, restore** — как обеспечиваем сохранность
- **Мониторинг, алерты, SLO** — как замечаем нарушение результата · **масштабирование, capacity, стоимость** — как выбираем ресурсы

> **i Проверим границу руками**
> Один Terraform-проект создаст пять обычных VM для self-managed пути, второй — managed-кластер. После apply сравним, какие объекты видны в коде и в облачной консоли — и по какую сторону границы живёт каждый.

СОБЕС «Плюсы managed: меньше своей работы по control plane и etcd, за них отвечает провайдер по SLA. Минусы: меньше контроля — на эти узлы не зайти, версии и флаги в темпе провайдера, зависимость от его возможностей. Приложение, данные, RBAC, мониторинг и стоимость всё равно мои. Managed не значит „за всё отвечает облако“ — границу и SLA проверяю у конкретного сервиса».

Что выбираем и почему — до первого запуска cluster.yml

Задача: состав стенда выбирается до установки и записывается в group_vars. Стенд учебный: функции совмещены на трёх VM, цена — общие ресурсы и отказы; production считает изоляцию и зоны отказа отдельно.

РЕШЕНИЕ	ЧТО ВЫБРАЛИ	ПОЧЕМУ
Версия	Kubernetes `1.34.7`, Kubespray `v2.31.0`	два разных продукта; версия из матрицы release; 1.35.4 — для upgrade позже
ОС нод	Ubuntu 24.04, user `ubuntu`	образ `ubuntu-2404-lts` из Terraform; Kubespray ставит всё поверх
Runtime	`containerd`	kubelet говорит с ним через CRI; Docker Engine не нужен
CNI	`calico`	IPAM выдаёт Pod IP, маршруты между нодами, NetworkPolicy; замена = миграция сети
kube-proxy mode	`iptables`, явно	в sample v2.31.0 стоит IPVS, а в k8s 1.35 IPVS deprecated; правила в ядре ноды
Сети	VM `10.10.0.0/24`, Service `10.233.0.0/18`, Pod `10.233.64.0/18`	три непересекающихся пула; /18 — запас из sample; сверить с сетями офиса/VPN
etcd	3 участника, `host`	кворум 2 из 3 — один отказ переживём; бинарник под systemd — штатный вариант release
DNS	CoreDNS + NodeLocal DNSCache	имя Service вместо IP; кэш на ноде отвечает рядом

СЛОЙ ПЛАТФОРМЫ	СЕЙЧАС	ПОЧЕМУ НЕ СЕЙЧАС
Ingress / Gateway API	не ставим	первый запрос — изнутри; YAML Ingress сам трафик не обслуживает, нужна реализация
LoadBalancer	не ставим	внешний адрес — отдельный вопрос; lb1/lb2 пока Linux-машины
Storage · Addons	не подключаем	база к первому тесту не подключена; metrics-server, cert-manager — по задаче

LOCAL `07_1.5_kubespray_full/inventory/video2/group_vars — те же решения строками`

```
kube_version: 1.34.7                kube_network_plugin: calico
container_manager: containerd       # kubelet → CRI → containerd
kube_proxy_mode: iptables           kube_proxy_strict_arp: false # ARP: IPVS
kube_service_addresses: 10.233.0.0/18      kube_pods_subnet: 10.233.64.0/18
kube_network_node_prefix: 24        # node CIDR k8s, не block size Calico IPAM
cluster_name: cluster.local  dns_mode: coredns  enable_nodelocaldns: true
etcd_deployment_type: host          # all/etcd.yml — прочитает роль etcd
```

СОБЕС «До установки фиксирую версию из матрицы установщика, runtime (containerd через CRI), CNI (Calico), kube-proxy явно (iptables — sample даёт IPVS), три непересекающихся CIDR и etcd: три участника, кворум два, host под systemd. Всё это — строки group_vars, которые прочитают роли».

Пять VM: три в кластер, две — на внешний вход

План стенда на два выпуска. Пять VM в одной приватной подсети `10.10.0.0/24`: три станут Kubernetes-кластером, две останутся отдельными Linux-машинами. Создаём все пять одним Terraform-проектом; кластер сегодня ставим только на node1–node3.

HOSTNAME	PUBLIC IP	PRIVATE IP	РОЛЬ	CPU / RAM / ДИСК	ЧТО НА МАШИНЕ
`node1`	<NODE1_IP>	<NODE1_PRIVATE_IP>	control-plane + etcd	4 / 8 GB / 40 GB	api-server, scheduler, controller-manager (static Pod), etcd (systemd); без приложений
`node2`	<NODE2_IP>	<NODE2_PRIVATE_IP>	control-plane + etcd + worker	4 / 8 GB / 40 GB	2-й control-plane, 2-й etcd, kubelet + containerd, поды приложений
`node3`	<NODE3_IP>	<NODE3_PRIVATE_IP>	worker + etcd	2 / 4 GB / 30 GB	3-й etcd, kubelet + containerd, поды приложений
`lb1`	<LB1_IP>	<LB1_PRIVATE_IP>	вне Kubernetes	2 / 2 GB / 20 GB	Linux-машина под внешний HA-вход — следующая часть
`lb2`	<LB2_IP>	<LB2_PRIVATE_IP>	вне Kubernetes	2 / 2 GB / 20 GB	вторая машина того же слоя

i Общее для всех VM

Ubuntu 24.04 (образ семейства `ubuntu-2404-lts`), SSH-ключ, user `ubuntu`, диск network-ssd, внешний IP через `nat: true`. Размеры — учебные, не production-нормы. Управление и SSH разрешены только с одного доверенного внешнего адреса `<YOUR_IP>/32` — адрес, с которым твоё подключение приходит в облако, не IP ноды и не домашний 192.168.x.x.

⚠ Почему etcd три, а не четыре — и почему это не HA-вход

Запись подтверждает большинство: у трёх — два, один отказ переживаем. У четырёх нужны три голоса, допустимый отказ всё равно один. Для двух отказов — пять участников. API-серверов здесь два, но адрес в kubeconfig принадлежит node1: при его отказе на node2 ничего не переключится. Балансировку входа в API этой схемой не настроили.

СОБЕС «Учебный стенд: node1/node2 — control plane, etcd на трёх для кворума, node2 и node3 — worker. Совмещение экономит VM, но управление и приложения делят ресурсы и отказы — в production нагрузку отделяют и разносят по зонам отказа. lb1/lb2 в кластер не входят — это будущий внешний вход».

06 TERRAFORM СТЕНДА · 0/6 · ПОДГОТОВКА

Нулевой шаг: инструменты, вход в облако, ключ и свой адрес

Задача: до первой строчки кода стенда убедиться, что есть чем работать и от чьего имени. Дальше всё создаётся в облаке и тарифицируется, поэтому проверки делаем здесь.

LOCAL что должно быть установлено

```
terraform -version     # для проекта пяти VM: ⩾ 1.10, < 2.0
ansible --version      # ansible-core 2.18.x из .venv
yc version             # CLI Yandex Cloud
git --version ; python3 --version   # python 3.11–3.13 под .venv
```

LOCAL вход в облако и доступ для провайдера

```
yc init                # мастер: OAuth-токен, облако, каталог, зона
yc config profile list # какие профили уже есть

export YC_TOKEN="$(yc iam create-token)"
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
test -n "$YC_TOKEN"; echo $?        # 0 — переменная непустая
echo "len=${#YC_TOKEN}"             # ~350, сам токен не печатаем
```

ОЖИДАЕМЫЙ ВЫВОД

```
Your profile default Compute zone has been set to 'ru-central1-a'.
0
len=352
```

Читаем: сначала выполняется команда внутри `$( )`, её вывод попадает в переменную, а `export` отдаёт её Terraform. Без `export` провайдер скажет, что не получил данные для входа. Токен живёт около 12 часов.

Что не показывать на экране

Мастер попросит вставить OAuth-токен из браузера. Токен, страница входа и вывод `yc config list` открывают доступ к аккаунту целиком. Идентификаторы облака и каталога закрывать не нужно, это адреса, а не ключи.

LOCAL ключ для машин и свой внешний адрес

```
test -f "$HOME/.ssh/id_ed25519.pub"; echo $?   # 0 — ключ есть
ssh-keygen -t ed25519 -f "$HOME/.ssh/video2a_ed25519"  # если нужна своя пара
ssh-add "$HOME/.ssh/video2a_ed25519"           # если у ключа пароль
curl -4 -fsS https://api.ipify.org; echo       # адрес, который видит облако
```

Адрес пойдёт в `allowed_ssh_cidr` с суффиксом `/32`, то есть ровно один адрес. Через VPN он меняется при переподключении и доступ отваливается, это разобрано в Troubleshooting.

СОБЕС «Terraform не хранит секреты в коде. Токен получаю через CLI и отдаю провайдеру переменной окружения, поэтому `.tf` можно показывать и коммитить. Личное лежит в `tfvars`, которые не коммитятся».

06 TERRAFORM СТЕНДА · 1/5

Пять VM как код: root-модуль и карта файлов

Проект: `01_ЧАСТЬ_1_КЛАСТЕР/04_1.3_five_vm_terraform/terraform`. Это **root-модуль** — каталог, из которого запускаешь Terraform: все `.tf` в нём читаются как одна конфигурация, имена файлов — соглашение для человека.

ФАЙЛ	ЧТО ТАМ
`versions.tf` · `providers.tf`	Terraform `≥ 1.10, < 2.0`; провайдер `yandex-cloud/yandex` запинен на `0.219.0`. В `provider` только `zone` — токена в коде нет
`variables.tf`	`zone`, `cluster_name`, `vpc_cidr` = 10.10.0.0/24, размеры (control-plane 4 CPU/8 ГБ/40 ГБ, worker 2/4/30, lb 2/2/20), `ssh_user` = ubuntu, `allowed_ssh_cidr` — без default
`locals.tf`	карта машин `local.nodes`: роли, размеры, флаг `api_endpoint` (ниже)
`security.tf` · `network.tf`	один /32 → список `trusted_admin_cidrs` → модуль `./modules/network`: VPC, subnet, security group
`compute.tf`	`data` образа `ubuntu-2404-lts` + модуль `./modules/linux-vm`: вся карта `nodes`, subnet и SG из сетевого модуля, текст ssh-ключа
`outputs.tf` · `inventory.tf`	адреса, `ssh_commands`; три текстовых output: inventory Kubespray, `all.yml` с адресом API, inventory внешнего HA

LOCAL `locals.tf · карта машин — состав стенда задан один раз`

```
locals {
  nodes = {
    node1 = { roles = ["control-plane", "etcd"]
              resources    = var.control_plane_resources
              api_endpoint = true }     # через неё ходим к API с ноутбука
    node2 = { roles = ["control-plane", "etcd", "worker"]
              resources = var.control_plane_resources }
    node3 = { roles = ["etcd", "worker"], resources = var.worker_resources }
    lb1   = { roles = ["load-balancer"]
              resources = var.load_balancer_resources
              ha = { overlay_ip = "10.77.0.11", priority = 150 } }
    lb2   = { … как lb1: overlay_ip 10.77.0.12, priority 100 }  # внешний вход позже
  }
}
```

Читаем: слова в `roles` сами Kubernetes не ставят — это наши метки. Из них получатся и VM (по машине на ключ), и группы Ansible. Правка диска или сети — в одном месте, не в пяти копиях resource.

i Модуль — каталог с цельной задачей

Входные `variables`, выходные `outputs`, внутри — ресурсы. **network** = VPC + подсеть + фильтр; **linux-vm** = машины одного устройства по карте. Для маленькой одноразовой конфигурации оставить всё в root — тоже нормально.

СОБЕС «Root-модуль — каталог с `.tf`, Terraform читает их как одну конфигурацию. Состав стенда держу в одной карте `local.nodes` с ролями, модули подключаю явно через `module` + `source`. Модуль выделяю ради границы задачи, а не потому, что каждый resource обязан получить обёртку».

06 TERRAFORM СТЕНДА · 2/5

Security group: SSH и API только с твоего /32

Задача: дать доступ к стенду только себе. Security group — облачный firewall — пускает SSH (22) и Kubernetes API (6443) с одного адреса: твой публичный IPv4 с маской /32.

LOCAL адрес управления, ssh-ключ, env/video.tfvars

```
curl -4 -fsS https://api.ipify.org            # внешний IPv4 → allowlist
test -f "$HOME/.ssh/id_ed25519.pub"; echo $?   # 0 — есть; 1 — делаем пару
ssh-keygen -t ed25519 -f "$HOME/.ssh/video2a_ed25519"   # только если нет
test -f env/video.tfvars; echo $?              # 0 — уже есть; 1 — копируем
cp env/video.tfvars.example env/video.tfvars
# env/video.tfvars — правим две строки, остальное оставляем:
allowed_ssh_cidr    = "<YOUR_IP>/32"           # НЕ 203.0.113.42 из образца
ssh_public_key_path = "~/.ssh/id_ed25519.pub" # путь к .pub своей пары
```

ОЖИДАЕМЫЙ ВЫВОД

```
<YOUR_IP>                    # адрес ИСТОЧНИКА подключения, не адрес VM
0
```

Читаем: `allowed_ssh_cidr` — не подсеть 10.10.0.0/24 и не IP будущей VM, а адрес, с которого ты стучишься снаружи. В `variables.tf` у него нет default и есть `validation`: `can(cidrhost(…))` + запрет ровно `0.0.0.0/0` и `::/0`; проверки «только /32» нет — /32 ставишь сам. За VPN адрес выхода может отличаться: подтвердит только SSH.

LOCAL modules/network/main.tf · SG; trusted_admin_cidrs = [allowed_ssh_cidr]

```
resource "yandex_vpc_security_group" "this" {
  name       = "${var.name_prefix}-sg"
  network_id = yandex_vpc_network.this.id          # тот же VPC, что subnet
  ingress {                                        # SSH для Ansible
    protocol       = "TCP"
    port           = 22
    v4_cidr_blocks = var.trusted_admin_cidrs       # только твой /32
  }
  # ingress TCP 6443 (API) и 30000-32767 (NodePort) — тот же список /32
  ingress {                                        # ноды между собой — всё
    protocol          = "ANY"
    predefined_target = "self_security_group"
  }
  # egress ANY → 0.0.0.0/0 — машины качают пакеты и образы (исходящий!)
}
```

ingress здесь ≠ Kubernetes Ingress

Слово совпало, механизм другой: правило входящих соединений firewall. **egress 0.0.0.0/0** — исходящее, не «SSH всему интернету». `nat = true` даст внешний IP, но доступ закрыт, пока твой /32 не совпал.

СОБЕС «Доступ к стенду — allowlist по /32: SG пускает 22 и 6443 только с моего адреса, ноды общаются через `self_security_group`, наружу открыт egress. Переменная без default и с validation — случайный `0.0.0.0/0` не проедет. Сменился IP — правлю tfvars, plan/apply меняет только правила SG».

06 TERRAFORM СТЕНДА · 3/5

Одна VM × пять записей: for_each, диски, ключ — и apply

Задача: создать пять машин одним resource и применить план. Модуль `linux-vm` получает карту `nodes` и делает по VM на каждый ключ; проверяем конфигурацию, читаем план, потом применяем.

LOCAL compute.tf → modules/linux-vm/main.tf (сокращено)

```
module "linux_vm" {                                  # compute.tf
  source             = "./modules/linux-vm"
  nodes              = local.nodes                   # вся карта, с lb1/lb2
  image_id           = data.yandex_compute_image.ubuntu.id   # ubuntu-2404-lts
  subnet_id          = module.network.subnet_id      # output сетевого модуля
  security_group_ids = [module.network.security_group_id]
  ssh_public_key     = file(pathexpand(var.ssh_public_key_path))  # ТЕКСТ .pub
}
resource "yandex_compute_instance" "this" {    # modules/linux-vm/main.tf
  for_each    = var.nodes                            # VM на каждый ключ карты
  name        = "${var.cluster_name}-${each.key}"    # k8s-selfmanaged-node1
  resources { cores = each.value.resources.cores }   # memory — так же
  boot_disk { … image_id, size = resources.disk, type = "network-ssd" … }
  network_interface { subnet_id, nat = true, security_group_ids … }  # ext IP
  metadata = { ssh-keys = "${var.ssh_user}:${var.ssh_public_key}" }
  scheduling_policy { preemptible = false }          # непрерываемые VM
}
```

Читаем: ключи `for_each` стабильные — добавишь node4, индекс node2 не сдвинется (со списком и `count` сдвинулся бы). `metadata.ssh-keys` отдаёт образу пользователя и публичный ключ — пароля в Terraform нет. `data` ищет семейство образа, не пин: при повторном plan возможна замена `image_id`.

LOCAL доступ к облаку → init → validate → plan → apply

```
export YC_TOKEN="$(yc iam create-token)"      # IAM-токен живёт в сессии
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
yc resource-manager folder get --id "$YC_FOLDER_ID"   # имя каталога — наш?
terraform init                                # провайдер по lock-файлу
terraform validate                            # ждём Success
terraform plan -var-file=env/video.tfvars -out=video2.tfplan
terraform apply video2.tfplan                 # второго yes не будет
```

ОЖИДАЕМЫЙ ВЫВОД

```
Plan: 8 to add, 0 to change, 0 to destroy.    # сеть, подсеть, SG, пять VM
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.
```

Читаем: `YC_*` живут в этой сессии терминала — новый терминал или протухший токен → повторяешь export, а не ищешь ошибку в модулях. Зелёный plan не резервирует мощности: квота может упасть на apply. Упало частично — созданное не сносишь: первая ошибка → свежий plan.

СОБЕС «Пять VM — один resource с `for_each` по карте: ключ = имя машины, размеры из `each.value`. Ключ уезжает через `metadata.ssh-keys`, `nat = true` даёт внешний IP, `preemptible = false`. Цикл один: export `YC_*` → init → validate → plan -out → apply именно этот файл».

06 TERRAFORM СТЕНДА · 4/5

outputs → inventory: адреса из state, группы из ролей

Задача: Ansible нужны адреса машин. Руками не переписываем: они лежат в state, а `inventory.tf` собирает из них INI прямо в output — группы получаются из тех же ролей, по которым создавались VM.

LOCAL state — откуда берутся адреса

```
terraform state list                              # сеть, подсеть, SG, пять VM
terraform state show \
  'module.linux_vm.yandex_compute_instance.this["node1"]'
```

ОЖИДАЕМЫЙ ВЫВОД

```
name = "k8s-selfmanaged-node1"       id = "<INSTANCE_ID>"
    ip_address     = "<NODE1_PRIVATE_IP>"     # внутренний, 10.10.0.x
    nat_ip_address = "<NODE1_IP>"             # внешний — для SSH
```

Читаем: state — служебный JSON рядом с `.tf` (локальный — лабораторная схема; в команде — общий backend с блокировкой). Это сохранённое состояние, не проверка доступности: SSH докажет preflight.

LOCAL inventory.tf · output kubespray_inventory (сокращено)

```
output "kubespray_inventory" {
  value = <<-EOT
[all]
%{for name, node in module.linux_vm.nodes~}       # карта-РЕЗУЛЬТАТ модуля
%{if !contains(node.roles, "load-balancer")~}      # lb1/lb2 — мимо
${name} ansible_host=${node.external_ip} ip=${node.internal_ip} …
  … access_ip=${node.internal_ip} ansible_user=${var.ssh_user}  # одна строка
%{endif~}
%{endfor~}
[kube_control_plane]
%{for name, node in module.linux_vm.nodes~}
%{if contains(node.roles, "control-plane")~}${name}
%{endif~}%{endfor~}
# [etcd] — роль etcd; [kube_node] — worker; [k8s_cluster:children] — 2 группы
  EOT
}
```

Читаем: цикл идёт по `module.linux_vm.nodes` — **output** модуля (адреса созданных VM + роли), не по входу `local.nodes`. node2 попадёт в три группы — у него три роли. Output `kubespray_all_yml` кладёт внешний IP машины с `api_endpoint = true` в `supplementary_addresses_in_ssl_keys` — Kubespray включит его в сертификат API.

precondition: ровно одна машина с api_endpoint

`length(local.api_endpoint_nodes) == 1 && alltrue([… contains(roles, "control-plane")])`. Забыл флаг, поставил двум, выбрал worker — Terraform остановит на plan с `error_message`. Длина важна отдельно: для пустого списка `alltrue` вернул бы true.

СОБЕС «Inventory не пишу руками: адреса лежат в state, `inventory.tf` собирает INI циклом по output модуля, группы `kube_control_plane` / `etcd` / `kube_node` — по ролям. Адрес API выбираю флагом у одной control-plane-ноды, precondition ловит ошибку на plan».

06 TERRAFORM СТЕНДА · 5/5

Что получили: пять VM и один источник адресов

Задача: вывести outputs, сохранить inventory в файлы трёх потребителей и проверить, как Ansible их прочитал. Ни один IP руками не вводим.

LOCAL outputs → файлы → проверка структуры Ansible

```
terraform output nodes                          # 5 записей: адреса + roles
terraform output -raw kubespray_inventory \
  > ../../07_1.5_kubespray_full/inventory/video2/inventory.ini
terraform output -raw kubespray_all_yml \
  > ../../07_1.5_kubespray_full/inventory/video2/group_vars/all/all.yml
terraform output -raw kubespray_inventory \
  > ../../06_1.4bis_node_preflight/inventory/hosts.ini
terraform output -raw external_ha_inventory \
  > ../../13_1.11_external_ha/inventory/hosts.ini   # lb1/lb2 — внешний вход позже
"$KUBESPRAY_DIR/.venv/bin/ansible-inventory" --graph \
  -i ../../07_1.5_kubespray_full/inventory/video2/inventory.ini
```

ОЖИДАЕМЫЙ ВЫВОД

```
[all]
node1 ansible_host=<NODE1_IP> ip=<NODE1_PRIVATE_IP> access_ip=<NODE1_PRIVATE_IP>
ansible_user=ubuntu
node2 …   node3 …                              # lb1/lb2 отфильтрованы
[kube_control_plane] node1 node2   [etcd] node1 node2 node3   [kube_node] node2 node3
```

Читаем: `--graph`: @etcd — node1..3; @kube_control_plane — node1, node2; @kube_node — node2, node3; по SSH не ходит — это докажет preflight. `>` очищает файл ДО запуска команды: пустой файл — ошибка чтения output или не тот root, читай stderr.

VM	РОЛИ	PRIVATE IP	КУДА ПОПАЛА
`node1`	control-plane, etcd · `api_endpoint`	<NODE1_PRIVATE_IP>	kube_control_plane, etcd; IP → all.yml
`node2`	control-plane, etcd, worker	<NODE2_PRIVATE_IP>	kube_control_plane, etcd, kube_node
`node3`	etcd, worker	<NODE3_PRIVATE_IP>	etcd, kube_node
`lb1` · `lb2`	load-balancer	<LB1_PRIVATE_IP> · <LB2_PRIVATE_IP>	только external_ha_inventory

i sensitive и -raw

У outputs пяти VM пометки `sensitive` нет — адреса видны в общем `terraform output`. Видишь `<sensitive>` — старая пометка в state: правка `.tf` state не меняет, нужен plan/apply. `-raw` — строковые outputs без кавычек. State целиком не публикуешь.

СОБЕС «Terraform — единственный источник адресов: state → `output -raw` → файлы preflight, Kubespray и внешнего HA. Пересоздал VM — повторил экспорт. Итог: пять VM в 10.10.0.0/24, `ssh ubuntu@<IP>` по ключу, API через внешний адрес node1 — учебный одиночный endpoint, не HA».

07 MANAGED-КЛАСТЕР · 1/4

Managed: тот же язык, другие ресурсы

Второй кластер не собираем сами: описываем его тем же Terraform, а control plane и ноды поднимает облако. Проект: `01_ЧАСТЬ_1_КЛАСТЕР/05_1.4_managed_cluster/terraform`.

ФАЙЛ	ЧТО ТАМ
`main.tf`	пины (Terraform ≥ 1.9, yandex `0.219.0`), `provider`, все `variable`: VM 10.20.0.0/24, Pod 10.112.0.0/16, Service 10.96.0.0/16, `k8s_version` 1.34, STABLE, `node_count` 2, оба `allowed_*_cidr` — без default
`network.tf`	VPC, subnet, SG: `self_security_group`, health-check LB, 443/6443 из `allowed_api_cidr`, NodePort/ICMP из `allowed_nodeport_cidr`
`service-accounts.tf`	два сервисных аккаунта и три роли (ниже)
`kubernetes.tf`	`yandex_kubernetes_cluster` — master в одной зоне (стр. 2/4) + `yandex_kubernetes_node_group` workers (стр. 3/4)
`outputs.tf`	`cluster_id`, `cluster_endpoint` (sensitive), `k8s_version`, `get_credentials_command`, `cidrs`
`*.auto.tfvars`	читаются сами, без `-var-file`; личные значения (оба CIDR /32) — в `private.auto.tfvars`, он в `.gitignore`

LOCAL service-accounts.tf · два аккаунта, три роли (сокращено)

```
resource "yandex_iam_service_account" "cluster" {   # 1) SA самого кластера:
  name = "${var.cluster_name}-cluster-sa"           #    LB, диски, SG
}
resource "yandex_iam_service_account" "nodes" {     # 2) SA узлов:
  name = "${var.cluster_name}-nodes-sa"             #    образы из registry
}
data "yandex_client_config" "client" {}             # folder_id — из окружения
resource "yandex_resourcemanager_folder_iam_member" "cluster_editor" {
  folder_id = data.yandex_client_config.client.folder_id
  role      = "k8s.clusters.agent"                  # ресурсы кластера
  member    = "serviceAccount:${yandex_iam_service_account.cluster.id}"
}
# cluster_lb   → role "vpc.publicAdmin"                (публичные IP для LB)
# nodes_puller → role "container-registry.images.puller" (чтение образов)
```

Читаем: самая частая точка отказа managed. Аккаунтов **два**, роли разные: перепутал — кластер не создастся или не сможет тянуть образы. `member` ссылается на `.id` аккаунта выше — так Terraform знает порядок.

⚠ **depends_on задаёт порядок, но IAM асинхронный**

В `kubernetes.tf` кластер стоит `depends_on` на три `iam_member`: сначала права, потом кластер. Но это порядок операций, не гарантия, что роль «прожалась»: apply может упасть с `permission denied`.

СОБЕС «Managed описываю тем же Terraform: VM 10.20.0.0/24, Pod 10.112.0.0/16, Service 10.96.0.0/16 — три непересекающихся диапазона; в self-managed Pod/Service CIDR задаёт Kubespray. Два сервисных аккаунта: кластера — `k8s.clusters.agent` + `vpc.publicAdmin`, узлов — `images.puller`. Кластер зависит от ролей через depends_on, но IAM асинхронный».

07 MANAGED-КЛАСТЕР · 2/4

Запуск managed: apply работает, мы читаем kubernetes.tf

Задача: в новом терминале получить свежий доступ к облаку, проверить план на 10 ресурсов и запустить apply; пока облако работает — читаем описание кластера.

LOCAL терминал 1 · managed-проект: доступ → init → validate → plan → apply

```
cd 01_ЧАСТЬ_1_КЛАСТЕР/05_1.4_managed_cluster/terraform
yc config profile list                          # выбран учебный профиль?
YC_TOKEN="$(yc iam create-token)" && export YC_TOKEN   # токен не печатаем
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
yc resource-manager folder get --id "$YC_FOLDER_ID"   # наш каталог?
# private.auto.tfvars: оба *_cidr = твой ТЕКУЩИЙ IP/32 — сверь перед plan
terraform init                                  # отдельный root, свой provider
terraform validate
terraform plan -out=managed.tfplan              # *.auto.tfvars читаются сами
terraform apply managed.tfplan                  # yes не спросит
```

ОЖИДАЕМЫЙ ВЫВОД

```
Plan: 10 to add, 0 to change, 0 to destroy.
# VPC, subnet, SG, 2 service account, 3 IAM-member, cluster, node group
yandex_kubernetes_cluster.this: Creating...
yandex_kubernetes_cluster.this: Still creating... [5m0s elapsed]
```

Читаем: worker-машин в плане нет — их ведёт node group. `Still creating` минутами — норма: control plane на записи поднялся за семь минут. Второй apply поверх работающего не запускаем.

LOCAL kubernetes.tf · resource yandex_kubernetes_cluster (сокращено)

```
resource "yandex_kubernetes_cluster" "this" {
  master {
    version = var.k8s_version                       # "1.34"
    master_location { zone = var.zone … }           # ОДНА зона: учебный стенд,
    public_ip          = true                       # не защита от отказа зоны;
    # kubectl снаружи, но SG фильтрует API по allowed_api_cidr
    security_group_ids = [yandex_vpc_security_group.k8s.id]
    maintenance_policy { auto_upgrade = false }     # в RAPID не выключить
  }
  cluster_ipv4_range = var.cluster_ipv4_range       # Pod     10.112.0.0/16
  service_ipv4_range = var.service_ipv4_range       # Service 10.96.0.0/16
  service_account_id      = yandex_iam_service_account.cluster.id
  node_service_account_id = yandex_iam_service_account.nodes.id
  release_channel = var.release_channel             # STABLE — потом не сменить
  depends_on = [ …три iam_member… ]                 # сначала права
}
```

СОБЕС «Managed запускаю тем же циклом: init → validate → plan -out → apply; в плане 10 ресурсов, worker-VM среди них нет — их ведёт node group. Master в одной зоне с `public_ip` для kubectl, auto_upgrade выключен, Pod/Service CIDR отдаю в API облака».

07 MANAGED-КЛАСТЕР · 3/4

Worker-группа и outputs: что облако отдаст назад

Задача: дочитать второй resource — группу рабочих нод — и понять, какие outputs заберём после apply. Читаем, пока в терминале 1 идёт `Still creating`.

LOCAL kubernetes.tf · resource yandex_kubernetes_node_group (сокращено)

```
resource "yandex_kubernetes_node_group" "workers" {
  cluster_id = yandex_kubernetes_cluster.this.id   # ссылка = зависимость
  instance_template {                              # каждая машина группы:
    platform_id = "standard-v3"                    # 2 CPU, 4 ГБ, nat = true,
    boot_disk { type = "network-ssd"  size = 30 }  # наша subnet + SG,
    container_runtime { type = "containerd" }      # preemptible = false
  }
  scale_policy {
    dynamic "fixed_scale" {                        # autoscaling=false → [1]
      for_each = var.enable_autoscaling ? [] : [1] # одна итерация
      content { size = var.node_count }            # = 2 ноды
    }
    dynamic "auto_scale" { for_each = var.enable_autoscaling ? [1] : [] … }
  }
  allocation_policy { location { zone = var.zone } }   # одна зона
  maintenance_policy { auto_upgrade = false  auto_repair = true }
}
```

Читаем: `cluster_id` = тип.имя.id ресурса выше — по ссылке Terraform понимает зависимость; версия группы не выше мастера. `dynamic` — блок по условию: `for_each` из одного элемента даёт один вложенный блок, из пустого — ничего.

LOCAL терминал 1 · после Apply complete — outputs managed

```
terraform output cluster_id                  # sensitive: по имени — покажет
terraform output cluster_endpoint
terraform output k8s_version
terraform output cidrs                       # три диапазона рядом
```

ОЖИДАЕМЫЙ ВЫВОД

```
Apply complete! Resources: 10 added, 0 changed, 0 destroyed.
"<CLUSTER_ID>"
"https://<CLUSTER_ENDPOINT>"                 # внешний адрес API-сервера
"1.34"
{ "nodes_subnet" = "10.20.0.0/24"  "pod_cidr" = "10.112.0.0/16"  "service_cidr" =
"10.96.0.0/16" }
```

Читаем: `cluster_id` и `cluster_endpoint` помечены `sensitive` — в общем `terraform output` скрыты, по имени отдаются. `get_credentials_command` — готовая строка `yc … get-credentials --id … --external`, выполним в разделе 09.

СОБЕС «Node group — второй resource, привязанный через `cluster_id = yandex_kubernetes_cluster.this.id`. Размер — dynamic-блоки: `for_each` по условию даёт либо `fixed_scale` с `node_count`, либо `auto_scale`. Worker-VM создаёт и чинит облако, но диски, размеры и зона — мои решения».

07 MANAGED-КЛАСТЕР · 4/4

Managed завис в Provisioning — разбираемся

Задача: apply в терминале 1 вышел по таймауту на группе нод: `context deadline exceeded`. Провайдер ждал час и перестал — это его лимит ожидания, не ответ облака. Спрашиваем само облако.

LOCAL терминал 1 · группа → узлы → квота → заявка

```
yc managed-kubernetes node-group list         # STATUS группы
yc managed-kubernetes node-group list-nodes --name k8s-managed-workers
yc quota-manager quota-limit get --quota-id compute.ssdDisks.size \
  --resource-type resource-manager.cloud --resource-id "$YC_CLOUD_ID"
# не хватает 10 ГиБ — просим 220 (236223201280 байт) с запасом
yc quota-manager quota-request create \
  --resource-type resource-manager.cloud --resource-id "$YC_CLOUD_ID" \
  --desired-limit quota-id=compute.ssdDisks.size,value=236223201280
yc quota-manager quota-request list \
  --resource-type resource-manager.cloud --resource-id "$YC_CLOUD_ID"
# APPROVED → группа сама повторит создание; через 1–2 мин снова list-nodes
terraform plan -out=managed.tfplan   # ждём No changes: группа уже в state
```

ОЖИДАЕМЫЙ ВЫВОД

```
k8s-managed-workers   STATUS PROVISIONING   SIZE 2      node-1  READY
node-2   PROVISIONING / CREATING_INSTANCE   [RESOURCE_EXHAUSTED] The limit
         on total size of network-ssd disks has exceeded
limit: 214748364800   usage: 193273528320    # 200 ГиБ лимит, 180 занято
status: APPROVED                             # PENDING / PROCESSING — ждём
```

Читаем: считаем: пять VM стенда — 40+40+30+20+20 = 150 ГБ SSD, первая worker-нода — 30, итого 180; второй диск на 30 в лимит 200 не лезет. Control plane облако подняло само за семь минут, а диски нод — наша зона: размеры и квоты каталога задаём мы.

LOCAL запасной вариант · PENDING дольше 5 минут — одна нода

```
sed -i '' 's/^node_count *= *2/node_count = 1/' private.auto.tfvars
terraform plan -out=managed.tfplan     # workers: update in-place, size 2 → 1
terraform apply managed.tfplan         # destroy/replace в плане — стоп
yc managed-kubernetes node-group list  # STATUS RUNNING, SIZE 1
```

Читаем: приложение в managed не запускаем — одной ноды хватит. Одобрили — вернём `node_count = 2` одним plan/apply. `sed -i ''` — macOS.

Provisioning ничего не объясняет

Статус группы — не диагноз. Диагноз у **узла**: `list-nodes` показывает каждому свой статус и сообщение — там и лежит `RESOURCE_EXHAUSTED`. Таймаут провайдера — не отказ облака.

СОБЕС «Managed тоже упирается в квоты каталога: control plane облако подняло само, а второй worker встал на `compute.ssdDisks.size` — диски нод задаю я. Диагностика: `node-group list` → `list-nodes` (сообщение у узла) → `quota-limit get`. Фикс: `quota-request create` с запасом или уменьшаю `node_count`; одобрили — возвращаю одним plan/apply».

08 УСТАНОВКА · 1/6 · PREFLIGHT

Preflight: проверяем Linux до запуска установщика

Задача: до длинного прогона Kubespray убедиться, что до node1–node3 есть SSH, sudo отдаёт root, а ОС, Python, swap, модули ядра и ip_forward в нужном состоянии. Проверяем с того же ноутбука и тем же Ansible из `.venv` клона, которыми запустим установку: красная ошибка здесь дешевле, чем внутри cluster.yml.

LOCAL 06_1.4bis_node_preflight · доступ, sudo, проверки Linux

```
cd 01_ЧАСТЬ_1_КЛАСТЕР/06_1.4bis_node_preflight
export KUBESPRAY_DIR="$(cd ../03_1.2bis_kubespray_anatomy/kubespray \
  && pwd -P)"                          # ansible берём из .venv клона

# ansible.cfg этой папки: inventory/hosts.ini + roles — флаги не нужны
"$KUBESPRAY_DIR/.venv/bin/ansible-inventory" --graph       # кто в группах
"$KUBESPRAY_DIR/.venv/bin/ansible" all -m ping             # SSH + Python
"$KUBESPRAY_DIR/.venv/bin/ansible" all --become -a 'whoami' # sudo → root
"$KUBESPRAY_DIR/.venv/bin/ansible-playbook" playbooks/preflight.yml
```

ОЖИДАЕМЫЙ ВЫВОД

```
@k8s_cluster: @kube_control_plane → node1, node2 · @kube_node → node2, node3
node1 | SUCCESS => {"changed": false, "ping": "pong"}     # + node2, node3
node1 | CHANGED | rc=0 >> root                              # + node2, node3
PLAY RECAP → node1 : ok=7 changed=0 unreachable=0 failed=0   # + node2, node3
```

Читаем: `-m ping` — не ICMP: SSH + Python на той стороне. Жёлтый `CHANGED` у `whoami` — не изменение: модуль command не знает желаемого состояния. Роль `node_preflight` только читает (`changed_when: false`) и assert'ит: Ubuntu/Debian, Python 3, пустой `swapon`, `overlay` и `br_netfilter` в `lsmod`, `ip_forward = 1`. `failed=0` у трёх нод — ставим.

LOCAL только если assert красный на модулях или forwarding

```
"$KUBESPRAY_DIR/.venv/bin/ansible-playbook" \
  playbooks/prepare-network.yml --list-hosts   # ровно node1-3? иначе стоп
"$KUBESPRAY_DIR/.venv/bin/ansible-playbook" \
  playbooks/prepare-network.yml                # modules-load.d + sysctl.d
"$KUBESPRAY_DIR/.venv/bin/ansible-playbook" \
  playbooks/preflight.yml                      # повтор: должен стать зелёным
```

i Kubespray закрыл бы эти три условия сам

`overlay` — `ExecStartPre=-modprobe overlay` в шаблоне `containerd.service.j2`; `br_netfilter` — роль `kubernetes/node`; `ip_forward` — `preinstall/0080-system-configurations.yml`. Готовим заранее: требования к ноде видны глазами, независимая проверка зелёная. SSH, sudo, ОС, Python и swap этот playbook не чинит — разбирай по сообщению.

СОБЕС «Перед Kubespray гоняю preflight тем же Ansible с той же машины: `-m ping` (SSH + Python), `--become whoami` (sudo → root), затем read-only playbook с assert на ОС, swap, `overlay` / `br_netfilter`, `ip_forward`. Красный assert — стоп-сигнал: читаю ноду и условие, чиню отдельным playbook, доказываю повторным зелёным прогоном».

08 УСТАНОВКА · 2/6 · INVENTORY И GROUP_VARS

inventory.ini и group_vars: что установщик прочитает

Задача: понять два набора входных данных Kubespray. **inventory.ini** — адреса и группы (его сгенерировал Terraform из outputs), **group_vars** — решения стенда: версия, runtime, CNI, диапазоны. Файлы лежат в `07_1.5_kubespray_full/inventory/video2/`, второго экземпляра Kubespray там нет.

ХОСТ	ANSIBLE_HOST	IP = ACCESS_IP	ГРУППЫ
node1	<NODE1_IP> — SSH снаружи	<NODE1_PRIVATE_IP>	kube_control_plane, etcd
node2	<NODE2_IP>	<NODE2_PRIVATE_IP>	kube_control_plane, etcd, kube_node
node3	<NODE3_IP>	<NODE3_PRIVATE_IP>	etcd, kube_node

Читаем: у всех `ansible_user=ubuntu`. `[k8s_cluster:children]` = kube_control_plane + kube_node → node1–node3: по этой группе идут и preflight, и play'и установки. etcd на трёх — большинство 2 из 3, одна нода может отказать. node2 совмещает роли, чтобы уместить стенд в три машины; для production размещение планируют отдельно.

LOCAL `inventory/video2/group_vars — только решения этого стенда`

```
# k8s_cluster/k8s-cluster.yml — каталог = имя группы из inventory
kube_version: 1.34.7              # из матрицы release v2.31.0; upgrade → 1.35
kube_network_plugin: calico       # Pod IP + маршруты между нодами
container_manager: containerd     # runtime на всех нодах и control plane
kube_proxy_mode: iptables         # strict_arp: false — он для IPVS
kube_service_addresses: 10.233.0.0/18
kube_pods_subnet: 10.233.64.0/18  # Service и Pod — разные диапазоны
kube_network_node_prefix: 24
kubeconfig_localhost: true        # admin.conf → artifacts/ на машине запуска
kubeconfig_localhost_ansible_host: true   # server = внешний адрес node1
# all/etcd.yml
etcd_deployment_type: host        # бинарник etcd под systemd, не Pod
# all/all.yml — Terraform подставил public IP node1 для TLS SAN
supplementary_addresses_in_ssl_keys: [<NODE1_IP>]
```

Классика: перепутать ansible_host и ip/access_ip

`ansible_host` — внешний адрес, по нему заходит SSH. `ip` / `access_ip` — внутренний: его займут etcd и kubelet, по нему ноды ищут друг друга. Внешний адрес в облаке приходит через NAT и на интерфейсе машины отсутствует. Попадёт во внутренние поля — etcd попытается занять адрес, которого нет, и установка встанет на первом же сервисе.

СОБЕС «Kubespray читает inventory (машины и группы `kube_control_plane` / `etcd` / `kube_node`) и group_vars по соглашению Ansible: каталог с именем группы. В sample сотни переменных, в моём файле — только решения стенда: `kube_version`, `container_manager: containerd`, `kube_network_plugin: calico`, `kube_proxy_mode: iptables`, непересекающиеся CIDR Service/Pod, `etcd_deployment_type: host`».

08 УСТАНОВКА · 3/6 · ЗАПУСК

cluster.yml: подключаем конфигурацию к клону и запускаем

Задача: положить наш `inventory/video2` внутрь клона Kubespray отдельным каталогом (sample не трогаем) и запустить корневой `cluster.yml` — ту обёртку с одним `import_playbook`, которую читали в разделе 05.

LOCAL клон v2.31.0 и .venv — если ещё не сделал в разделе 05

```
cd 01_ЧАСТЬ_1_КЛАСТЕР/03_1.2bis_kubespray_anatomy
git clone --branch v2.31.0 --depth 1 \
  https://github.com/kubernetes-sigs/kubespray.git
cd kubespray && git describe --tags --exact-match       # → v2.31.0
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/ansible-playbook --version                      # core 2.18.x
```

LOCAL корень клона · inventory/video2 → cluster.yml

```
mkdir -p inventory/video2
cp -R ../../07_1.5_kubespray_full/inventory/video2/. inventory/video2/
# «/.» = содержимое каталога: повтор обновит те же файлы, без video2/video2
.venv/bin/ansible-inventory -i inventory/video2/inventory.ini --graph
.venv/bin/ansible-playbook -i inventory/video2/inventory.ini \
  --become cluster.yml                    # 30–60 минут, терминал не закрывать
```

ОЖИДАЕМЫЙ ВЫВОД (ПЕРВЫЕ МИНУТЫ)

```
@kube_control_plane: node1, node2 · @kube_node: node2, node3 · @etcd: все три

PLAY [Check Ansible version] ***************************************
ok: [localhost]                          # служебная задача на машине запуска
PLAY [...] ***
TASK [kubernetes/preinstall : ...] ***    # роль : задача = где мы сейчас
ok: [node1]
changed: [node2]
changed: [node3]
```

Читаем: сначала Kubespray проверяет версию Ansible, дальше заголовки `PLAY` называют этапы, `TASK` — конкретные задачи, а node1–node3 появляются в строках результатов. Строки с `localhost` — работа на твоей машине. Ждём `PLAY RECAP` (6/6); пока идёт — разбираем этапы и роли.

⚠ **Встало на проверке версии Ansible**

v2.31.0 требует **ansible-core ≥ 2.18.0 и < 2.19.0**. Сверь первую строку `--version` (`core 2.18.x`) и что executable ведёт в `.venv` этого клона. Пакет `ansible 11.x` из requirements — не номер core. Проверку из Kubespray не вырезаем; другая ошибка — имя задачи, host, msg.

СОБЕС «Ставлю Kubespray с закреплённого release-tag, а не с main; зависимости — в `.venv` клона, чтобы не ломать системный Ansible. Свой inventory и group_vars держу отдельным каталогом внутри клона, sample не правлю. Запуск — один `ansible-playbook -i … --become cluster.yml`; дальше читаю PLAY/TASK и жду PLAY RECAP».

08 УСТАНОВКА · 4/6 · СЕМЬ СТАДИЙ

Что делает Kubespray: семь стадий cluster.yml

Этапы идут в порядке play'ев `playbooks/cluster.yml`. Имя роли в строке `TASK [роль : задача]` говорит, на каком этапе установка сейчас. Никакой магии — обычные Ansible-роли из каталога `roles/` клона.

#	СТАДИЯ · РОЛЬ	HOSTS	ЧТО ПОЯВЛЯЕТСЯ НА НОДАХ
1	**preinstall + download** `kubernetes/preinstall`, `download`	k8s_cluster	пакеты, sysctl (ip_forward), бинарники и образы скачиваются **на каждую ноду**. Упала загрузка — адрес и сообщение в терминале
2	**container runtime** `container-engine/containerd`	k8s_cluster	containerd: получает образы и запускает контейнеры по запросу kubelet. Выбран через `container_manager`
3	**etcd** `etcd` → install_host.yml	etcd: node1–3	бинарник, `/etc/etcd.env`, сертификаты, systemd-сервис etcd — хранилище данных API
4	**node** `kubernetes/node`	k8s_cluster	kubelet на всех нодах: бинарник, kubelet.env, unit, br_netfilter. Идёт **раньше** control plane
5	**control plane** `kubernetes/control-plane`	kube_control_plane: node1, node2	kubeadm init → файлы в `/etc/kubernetes/manifests` → kubelet запускает apiserver, scheduler, controller-manager как static Pod
6	**kubeadm join + CNI** `network_plugin/calico`	k8s_cluster	ноды входят в кластер; Calico выдаёт адреса из Pod-диапазона и строит маршруты. До этого ноды в кластере есть, но `NotReady`
7	**apps** `kubernetes-apps`	control plane	CoreDNS (Deployment) отвечает на имена Service, NodeLocal DNS — кэш на нодах

LOCAL как читать терминал во время установки

```
TASK [kubernetes/node : Install kubelet binary] ****   # роль : задача
ok: [node1]                        # состояние уже нужное, ничего не менял
changed: [node2]                   # задача что-то изменила на ноде
fatal: [node3]: FAILED! => {"msg": "..."}   # host + msg; установка стоит
fatal: [node3]: UNREACHABLE!       # SSH до ноды пропал — см. раздел 12
```

i Где это в репозитории

Play'и — `playbooks/cluster.yml` (корневой `cluster.yml` только импортирует его). Роли — `roles/container-engine/`, `roles/etcd/`, `roles/kubernetes/{preinstall,node,control-plane}/`, `roles/network_plugin/`. Умолчания и проверки версий — `roles/kubespray_defaults/`.

СОБЕС «Kubespray — последовательность play'ев: подготовка ОС и download → containerd → etcd (у меня host, systemd) → kubelet на всех нодах → control plane через kubeadm как static Pod на node1/node2 → join + Calico, после чего ноды становятся Ready → CoreDNS. По имени роли в TASK я всегда знаю, где остановилась установка».

08 УСТАНОВКА · 5/6 · НАСТОЯЩИЕ ЗАДАЧИ

Роли изнутри: откуда берутся kubelet, etcd и static Pod

Задача: проследить, как наши значения из group_vars доходят до конкретных задач в клоне `kubespray/` (v2.31.0), и что эти задачи реально делают на нодах. Открывай файлы в VS Code параллельно с бегущей установкой.

НАШЕ ЗНАЧЕНИЕ	ГДЕ ЧИТАЕТСЯ В КЛОНЕ	ЧТО ПОЛУЧАЕТСЯ
`container_manager: containerd`	`roles/container-engine/tasks/main.yml` — словарь `container_manager_role` + `include_role`	подключается роль `container-engine/containerd`
`kube_network_plugin: calico`	`roles/network_plugin/tasks/main.yml`: `include_role name: "network_plugin/{{ kube_network_plugin }}"`	имя роли `network_plugin/calico`
`etcd_deployment_type: host`	`roles/etcd/tasks/main.yml` Install etcd → `include_tasks install_host.yml`; шаблон `etcd-host.service.j2`	unit с `EnvironmentFile=/etc/etcd.env` и `ExecStart` на бинарник — systemd ведёт etcd на трёх нодах
группа `kube_control_plane`	play **Install the control plane** → роль `kubernetes/control-plane` → `kubeadm-setup.yml`	`kubeadm-config.yaml` из шаблона → `kubeadm init` на первой control-plane ноде

LOCAL `roles/kubernetes/node — путь kubelet: бинарник → unit → сервис (выжимка)`

```
# tasks/install.yml — бинарник уже лежит на ноде: его положил этап download
- name: Install kubelet binary
  copy: src={{ downloads.kubelet.dest }} dest={{ bin_dir }}/kubelet
        remote_src=true mode=0755        # remote_src: брать с самой ноды
# tasks/kubelet.yml — шаблон читает Ansible, готовый unit читает systemd
- template: src=kubelet.service.j2 dest=/etc/systemd/system/kubelet.service
- service:  name=kubelet enabled=true state=started   # Enable kubelet
# templates/kubelet.service.j2 — нужные строки
After={{ container_manager }}.service    # у нас: после containerd.service
EnvironmentFile=.../kubelet.env          # параметры из отдельного шаблона
ExecStart={{ bin_dir }}/kubelet ...      # Restart=always; WantedBy=[Install]
```

i Почему kubelet на всех нодах, а API-сервер — нет

Play **Install Kubernetes nodes** идёт по `k8s_cluster`: kubelet — единственный, кто просит containerd запускать контейнеры, без него нода не запускает Pod. Play **Install the control plane** идёт только по `kube_control_plane` (node1, node2). Там `kubeadm init` кладёт описания apiserver, scheduler и controller-manager в `/etc/kubernetes/manifests`; kubelet читает эти файлы и запускает их как **static Pod** — поэтому control plane поднимается до того, как через API начнут создавать что-либо. Когда API доступен, kubelet публикует туда **mirror Pod** — отражение для наблюдения.

СОБЕС «Kubespray не магия поверх Ansible: play выбирает группу, роль получает её переменные, а значение `calico` или `containerd` через `include_role` выбирает нужные задачи. Итог на ноде: systemd запускает kubelet и etcd; kubelet через containerd запускает контейнеры; apiserver, scheduler и controller-manager — static Pod из `/etc/kubernetes/manifests`, которые создал kubeadm».

08 УСТАНОВКА · 6/6 · PLAY RECAP

PLAY RECAP: читаем итог и что делать, если не взлетело

Задача: прочитать сводку Ansible по каждой ноде и решить, можно ли считать установку законченной. Это отчёт установщика о своих задачах; ответ самого Kubernetes получим в разделе 09 через kubectl.

LOCAL хвост cluster.yml — сводка по нодам

```
# установка закончилась: 30-60 минут в зависимости от машин, сети и образов
# проверяем, что в таблице есть node1, node2, node3 — и читаем счётчики
```

ОЖИДАЕМЫЙ ВЫВОД

```
PLAY RECAP *********************************************************
localhost: ok=… changed=… unreachable=0 failed=0 skipped=… rescued=0 ignored=0
node1    : ok=… changed=… unreachable=0 failed=0 skipped=… rescued=0 ignored=0
node2    : ok=… changed=… unreachable=0 failed=0 skipped=… rescued=0 ignored=0
node3    : ok=… changed=… unreachable=0 failed=0 skipped=… rescued=0 ignored=0
```

СЧЁТЧИК	ЧТО ЗНАЧИТ	НОРМА ПОСЛЕ ПЕРВОЙ УСТАНОВКИ
`ok`	успешные результаты задач	сотни на ноду
`changed`	результаты с изменением; уже посчитаны в ok	много — ожидаемо; при повторе прогона резко меньше
`unreachable`	нода не ответила по SSH	**0** у каждой ноды
`failed`	задача завершилась ошибкой	**0** у каждой ноды
`skipped`	задачи, пропущенные по условию (другой runtime, CNI, ОС)	много — это не ошибка
`rescued` / `ignored`	ошибка обработана блоком rescue / проигнорирована	0; если не 0 — найди, что произошло и как обработано

Читаем: `failed=0` и `unreachable=0` у трёх нод — Ansible выполнил все задачи. Это ещё не «кластер здоров»: готовность нод и системных Pod проверяем со стороны API (раздел 09).

✓ **Если не взлетело — коротко**

Мотай вверх до первого `fatal:`: там роль, задача, host и `msg`. **Download на одной ноде** (TLS handshake timeout к dl.k8s.io / github) — сеть конкретной VM наружу, а не Kubespray; повторный запуск той же команды `cluster.yml` идёт дальше: скачанное в `/tmp/releases` на ноде переиспользуется, checksum сверяется. **unreachable** посреди прогона — сменился твой публичный IP, security group пускает только `<YOUR_IP>/32`: обнови CIDR в tfvars, `terraform apply`, повтори playbook. **Проверка версии** — ansible-core 2.18.x. Подробно с командами — раздел 12.

СОБЕС «PLAY RECAP читаю по каждой ноде: `failed=0` и `unreachable=0` — обязательное условие, `changed` после первого прогона большой и входит в ok, `skipped` — нормальные ветки под другие runtime/CNI. Упало — иду к первому fatal, читаю роль, host и msg; Kubespray идемпотентен, поэтому после фикса просто повторяю тот же cluster.yml».

09 ДОСТУП · 1/4 · ДВА KUBECONFIG

Два кластера, один kubectl: сертификат в файле vs токен на лету

kubectl нужны адрес API-сервера, CA и способ представиться. Это kubeconfig: **clusters** (server + CA), **users** (как аутентифицируется клиент), **contexts** (кластер + пользователь). Контекстом переключаемся между `kubespray` и `k8s-managed`; разница кластеров — целиком в users.

	KUBESPRAY · SELF-MANAGED	K8S-MANAGED · YANDEX
Откуда файл	его подготовила установка: `inventory/video2/artifacts/admin.conf`	`yc managed-kubernetes cluster get-credentials` вписывает всё сам
Что в users	`client-certificate-data` + `client-key-data` — статичный сертификат с ключом в файле	`exec` → kubectl вызывает `yc` за IAM-токеном
Кто хранит личность	сам кластер: подписал своим CA — человек существует, пока сертификат не истёк	IAM облака: API-сервер проверяет токен и видит, кто пришёл
Срок	срок сертификата; списка отзыва у Kubernetes нет	токен живёт до 12 часов, в файле его нет
Утёк файл	утёк сам доступ: ждать истечения или перевыпускать CA — больно	без доступа к облачному аккаунту файл мало что даёт
Отзыв	удалить RoleBinding — прав нет, сертификат остаётся на руках	снять IAM-роль — следующий запрос отклонён, централизованно
Команда 10+ человек	OIDC на API-сервере (`--oidc-issuer-url`: Keycloak, GitLab), binding на группу из токена	уже сделано облаком за нас

LOCAL откуда взялся admin.conf — хвост k8s-cluster.yml + all.yml

```
kubeconfig_localhost: true                # admin.conf → на машину запуска
kubeconfig_localhost_ansible_host: true   # server = внешний адрес node1
kubectl_localhost: false                  # kubectl на ноутбуке уже есть
supplementary_addresses_in_ssl_keys: [<NODE1_IP>]  # иначе x509 на этот адрес
```

i Собес-вопрос: новому админу нужен доступ — выдать и отозвать

Доступ = **личность → права → файл**. Свой admin.conf не пересылаю: полный cluster-admin, не привязать и не отозвать. **Kubespray:** коллега делает ключ + CSR (CN = имя) → `CertificateSigningRequest` со сроком 30 дней → `kubectl certificate approve` → `clusterrolebinding --clusterrole=cluster-admin --user=ivan` → kubeconfig ему; проверка `auth can-i --as=ivan`; отзыв — `delete clusterrolebinding`. **Managed:** личность = аккаунт облака; `add-access-binding --role k8s.cluster-api.cluster-admin`; файл — сам через get-credentials; отзыв — `remove-access-binding`.

СОБЕС «kubectl один, кластер выбирается контекстом. В managed в users стоит `exec`: токен IAM приходит на лету и живёт часы, отзыв — в IAM. В своём кластере в файле лежит клиентский сертификат: утёк файл — утёк доступ, списка отзыва нет, защищает только срок. Для одного администратора — сертификат, для команды — OIDC».

09 ДОСТУП · 2/4 · SELF-MANAGED

admin.conf от Kubespray: первый вход и контекст kubespray

Задача: подключиться к собранному кластеру файлом, который подготовила установка, получить ответ от API (список нод) и дать контексту короткое имя. Работаем в корне клона Kubespray, установка завершена.

LOCAL корень клона · artifacts → первый запрос → контекст kubespray

```
ls inventory/video2/artifacts                          # admin.conf
kubectl --kubeconfig inventory/video2/artifacts/admin.conf \
  config get-contexts                                   # имена, без ключей
kubectl --kubeconfig inventory/video2/artifacts/admin.conf \
  get nodes                                             # ответ самого кластера
kubectl --kubeconfig inventory/video2/artifacts/admin.conf \
  config rename-context \
  kubernetes-admin-cluster.local@cluster.local kubespray
# учебная копия для этой сессии — общий ~/.kube/config не трогаем
cp inventory/video2/artifacts/admin.conf \
  inventory/video2/artifacts/recording.conf
chmod 600 inventory/video2/artifacts/recording.conf  # только владелец
export KUBECONFIG="${PWD}/inventory/video2/artifacts/recording.conf"
```

ОЖИДАЕМЫЙ ВЫВОД

```
admin.conf
CURRENT   NAME                                          CLUSTER
*         kubernetes-admin-cluster.local@cluster.local  cluster.local
NAME    STATUS   ROLES           AGE   VERSION
node1   Ready    control-plane   ..    v1.34.7
node2   Ready    control-plane   ..    v1.34.7
node3   Ready    <none>          ..    v1.34.7
Context "kubernetes-admin-cluster.local@cluster.local" renamed to "kubespray".
```

Читаем: API-сервер ответил по `https://<NODE1_IP>:6443` — TLS принят, администратор авторизован, три ноды Ready, VERSION ровно та, что в `kube_version`. `ROLES` — метки объекта Node от kubeadm: control-plane у node1/node2, `<none>` у node3. Pod приложения пойдут на node2 и node3 — группа `kube_node`. Имя контекста — из `cluster_name`; уже видишь `kubespray` с прошлого прохода — rename пропускай. `export KUBECONFIG` живёт только в этой вкладке.

Грабля: server в admin.conf смотрит «внутрь»

Kubespray с `kubeconfig_localhost_ansible_host: true` вписал внешний адрес node1, и он есть в SAN сертификата (`supplementary_addresses_in_ssl_keys`). Забираешь `/etc/kubernetes/admin.conf` с node1 руками — там внутренний адрес: с ноутбука `i/o timeout`. Подмени `server` на `https://<NODE1_IP>:6443`; нет адреса в SAN — `x509: certificate is valid for …`, чинится в сертификате, не в файле.

СОБЕС «После Kubespray admin.conf лежит в artifacts на машине запуска; в нём server — внешний адрес node1, который я заранее добавил в SAN. Файл не открываю и не пересылаю — это cluster-admin. Смотрю `config get-contexts`, спрашиваю кластер `get nodes`, переименовываю контекст в короткое имя и держу учебные контексты в отдельном KUBECONFIG с правами 600».

09 ДОСТУП · 3/4 · MANAGED

get-credentials: контекст k8s-managed и кто мы в каждом кластере

Задача: добавить managed-кластер в тот же recording.conf одной командой облачного CLI, увидеть оба контекста рядом и спросить каждый кластер, кем он нас считает. `KUBECONFIG` уже указывает на учебную копию.

LOCAL KUBECONFIG=recording.conf · оба контекста в одном файле

```
yc managed-kubernetes cluster get-credentials k8s-managed \
  --external --kubeconfig "$KUBECONFIG"          # внешний адрес API, наш файл
kubectl config rename-context \
  "$(kubectl config current-context)" k8s-managed   # yc назвал по ID
kubectl config get-contexts
kubectl get nodes                                  # текущий = k8s-managed!
kubectl config use-context kubespray               # умолчание — наш кластер

kubectl config view                                # ключи скрыты
kubectl --context k8s-managed auth whoami
kubectl --context kubespray auth whoami
```

ОЖИДАЕМЫЙ ВЫВОД

```
Context 'yc-managed-k8s-<CLUSTER_ID>' was added as current context.
Context "yc-managed-k8s-<CLUSTER_ID>" renamed to "k8s-managed".
CURRENT   NAME          CLUSTER
*         k8s-managed   yc-managed-k8s-<CLUSTER_ID>
          kubespray     cluster.local
NAME              STATUS   ROLES    VERSION    # только worker-ноды группы,
cl1…-<node>       Ready    <none>   v1.3x.x    # строк control-plane нет
Switched to context "kubespray".
users: k8s-managed → exec: command: yc …  |  kubernetes-admin → DATA+OMITTED
Username: <IAM_ACCOUNT_ID>                                  # k8s-managed
Username: kubernetes-admin   Groups: [kubeadm:cluster-admins …] # kubespray
```

Читаем: yc вписал адрес API и credentials, привязанные к твоей облачной учётке, и сделал контекст текущим — поэтому `get nodes` без флагов ушёл в облако. `config view` показывает разницу в users: `exec` с командой `yc` против `client-certificate-data: DATA+OMITTED`. `auth whoami`: managed видит ID твоего IAM-аккаунта, свой кластер — `kubernetes-admin` в группе `kubeadm:cluster-admins`.

i Граница managed в одной команде

В `get nodes` у k8s-managed только worker-ноды с именами от облака. Управляющие узлы обслуживает облако и объектов Node за них не выдаёт. В self-managed те же строки control-plane мы только что видели. Оба кластера — в одном файле, kubectl один и тот же; выбор — имя контекста.

СОБЕС «Для managed kubeconfig делает CLI облака: `get-credentials --external`, в users — exec, файл можно пересоздать одной командой. Держу оба контекста в одном KUBECONFIG, после yc возвращаю `use-context` на нужный кластер — иначе команда без флагов уйдёт не туда. `auth whoami` — быстрый способ показать, кто хранит личность: IAM или сертификат от CA кластера».

09 ДОСТУП · 4/4 · СИСТЕМНЫЕ КОМПОНЕНТЫ

Что установилось на самом деле: systemd, static Pod, Deployment

Задача: увидеть компоненты, чьи исходники читали в разделе 08: как сервисы Linux, как static Pod и как Deployment. Команды только читают состояние.

LOCAL корень клона · etcd и kubelet — сервисы systemd на нодах

```
.venv/bin/ansible -i inventory/video2/inventory.ini \
  etcd --become -m command -a 'systemctl is-active etcd'
.venv/bin/ansible -i inventory/video2/inventory.ini \
  k8s_cluster --become -m command -a 'systemctl is-active kubelet'
```

ОЖИДАЕМЫЙ ВЫВОД

```
node1 | CHANGED | rc=0 >> active        # + node2, node3 — для обоих сервисов
```

CLUSTER · KUBESPRAY kube-system: static Pod control plane, CoreDNS, runtime

```
kubectl --context kubespray -n kube-system get pods -o wide
kubectl --context kubespray -n kube-system \
  describe pod kube-apiserver-node1        # и -controller-manager, -scheduler
kubectl --context kubespray -n kube-system wait \
  --for=condition=Ready pod -l tier=control-plane --timeout=120s
kubectl --context kubespray -n kube-system \
  rollout status deployment/coredns --timeout=120s
kubectl --context kubespray get nodes -o wide   # CONTAINER-RUNTIME на всех
```

ОЖИДАЕМЫЙ ВЫВОД

```
NAME                           READY  STATUS   NODE
kube-apiserver-node1           1/1    Running  node1   # + -node2
kube-controller-manager-node1  1/1    Running  node1   # + scheduler; активен
kube-scheduler-node1           1/1    Running  node1   #  один из двух: лидер
Annotations: kubernetes.io/config.mirror: …   # static Pod → файл на ноде
deployment "coredns" successfully rolled out
node1  Ready  control-plane  …  containerd://1.x   # + node2, node3
```

Читаем: etcd и kubelet среди Pod искать не нужно — это systemd-сервисы (`etcd_deployment_type: host`). apiserver, scheduler, controller-manager — static Pod на node1/node2: в describe видна команда процесса, аннотация `config.mirror` = описание лежит файлом в `/etc/kubernetes/manifests`. CoreDNS — Deployment. CrashLoopBackOff или 0/1 — не здоров: describe + Events.

i В managed этого не видно

В kube-system у k8s-managed — coredns, kube-proxy, агенты облака; kube-apiserver/scheduler/controller-manager там нет. `yc compute instance list`: пять VM стенда и worker-ноды группы, машин control plane нет — ни SSH, ни systemctl.

СОБЕС «После Kubespray: etcd и kubelet — сервисы systemd (`systemctl is-active` через ansible по группам); apiserver, scheduler, controller-manager — static Pod из `/etc/kubernetes/manifests` с аннотацией `config.mirror`; CoreDNS — Deployment; runtime — containerd. В managed control plane не виден — это и есть граница ответственности».

10 ПРИЛОЖЕНИЕ В КЛАСТЕРЕ · 1/5

Namespace traffic-lab: пространство и правила безопасности

Задача: кластер отвечает — возвращаем DEVOPS MAY CRY. Каталог `01_ЧАСТЬ_1_КЛАСТЕР/08_1.6_verify_cluster/app`, шесть файлов. Читаем по порядку и ищем связи: namespace по имени, Pod по меткам, порт по имени `http`, общий список в kustomization. В Deployment — готовый образ из registry.

`1 namespace.yaml` `2 deployment.yaml` `3 service.yaml` `4 debug-client.yaml` `5 pdb.yaml` `6 kustomization.yaml`

LOCAL app/namespace.yaml — отдельный объект Namespace

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: traffic-lab                    # на это имя ссылаются остальные
  labels:
    app.kubernetes.io/part-of: devops-may-cry       # ярлык для фильтров
    pod-security.kubernetes.io/enforce: restricted  # не пускать
    pod-security.kubernetes.io/audit: restricted    # создать + в журнал
    pod-security.kubernetes.io/warn: restricted     # создать + предупредить
```

Читаем: `apiVersion` — версия API, `kind` — тип объекта, `metadata` — имя и метки, `spec` у Namespace не нужен. Три метки `pod-security.kubernetes.io/*` включают проверку каждого нового Pod по профилю **restricted** — самому строгому: контейнер не root, без права поднимать привилегии, без лишних прав ядра. Включается это метками на namespace, а не настройкой кластера.

i enforce / warn / audit — три реакции на одно нарушение

enforce — не пускать: Pod с нарушением не создастся. **warn** — создать, но написать предупреждение в kubectl. **audit** — создать и записать в журнал. Мы поставили все три: и не пускает, и видно, почему. Дальше в Deployment у контейнера будет блок `securityContext` — без этих меток он был бы хорошим тоном, с ними он обязателен: убери его — и `apply` ответит отказом.

⚠ Почему пока без базы

В Compose приложение работало с PostgreSQL. Здесь в Deployment нет `DB_HOST` — приложение стартует в режиме **Memory**: отвечает и считает заявки в памяти процесса. У двух реплик свои счётчики, после перезапуска счёт с нуля. Сначала проверяем запуск и путь запроса в новом кластере; базу подключим отдельно.

СОБЕС «Pod Security Admission включаю метками на namespace, не флагами кластера: **enforce** блокирует, **warn** предупреждает, **audit** пишет в журнал. Профиль **restricted** требует securityContext у каждого Pod — без него apply откажет. Так в namespace приложения никто случайно не запустит привилегированный контейнер».

10 ПРИЛОЖЕНИЕ В КЛАСТЕРЕ · 2/5

Deployment: selector, метки Pod и правила размещения

Задача: Deployment держит нужное число экземпляров через ReplicaSet и Pod: `replicas: 2`, план замены версии `RollingUpdate` — `maxSurge: 1` допускает одну лишнюю реплику, `maxUnavailable: 0` не даёт обновлению просесть ниже двух. Главный вопрос файла: как контроллер узнаёт свои Pod.

LOCAL app/deployment.yaml — фрагмент: selector, template, spec Pod

```yaml
kind: Deployment
metadata:
  name: devops-may-cry
  namespace: traffic-lab                # ссылка на Namespace по имени
spec:
  replicas: 2
  selector:
    matchLabels:
      app: devops-may-cry               # ← контроллер ищет Pod с этой меткой
  template:                             # описание будущего Pod
    metadata:
      labels:
        app: devops-may-cry             # ← и ставит её на каждый новый Pod
    spec:                               # настройки Pod — ещё до контейнера
      terminationGracePeriodSeconds: 30  # SIGTERM, ждать до 30 с, потом убить
      automountServiceAccountToken: false  # токен API в контейнер не класть
      securityContext:                  # для всех контейнеров Pod сразу
        runAsNonRoot: true
        seccompProfile: {type: RuntimeDefault}
      topologySpreadConstraints:        # где размещать, а не сколько
        - maxSkew: 1                    # разница между нодами не больше 1
          topologyKey: kubernetes.io/hostname   # сравнивать ноды по имени
          whenUnsatisfiable: ScheduleAnyway     # мягко: не вышло — куда можно
          labelSelector:
            matchLabels: {app: devops-may-cry}  # какие Pod считать
```

Читаем: `selector.matchLabels` и `template.metadata.labels` — одна метка с двух сторон. Контроллер не помнит Pod по именам: он ищет всё, что помечено `app=devops-may-cry`, а через template ставит эту метку на каждый новый Pod. Поэтому «дублирование» обязательно — это и есть связь контроллера с его Pod. Ту же метку через минуту увидим в Service.

i template.spec — три строки, которые требует restricted

`automountServiceAccountToken: false` — по умолчанию каждый Pod получает токен API файлом; нашему приложению API не нужен, а лишний токен — лишний способ навредить, если приложение взломают. `runAsNonRoot` — иначе Pod не стартует. `seccompProfile: RuntimeDefault` — стандартный профиль syscall'ов, отрезает то, что обычному приложению не нужно.

СОБЕС «Deployment находит свои Pod не по именам, а по **selector.matchLabels**; **template.metadata.labels** ставит ту же метку на каждый созданный Pod — одна связь с двух сторон, и Service потом цепляется за неё же. Размещение по нодам задаю **topologySpreadConstraints**: maxSkew 1 по hostname, **ScheduleAnyway** — предпочтение, не запрет: если развести нельзя, обе реплики встанут на одну ноду».

LOCAL app/deployment.yaml — template.spec.containers

```yaml
containers:
  - name: devops-may-cry
    image: ghcr.io/boost-mentor/devops-may-cry@sha256:<DIGEST>  # digest
    imagePullPolicy: IfNotPresent
    ports:
      - name: http                    # по имени ссылаются probes и Service
        containerPort: 8080           # здесь слушает приложение
    startupProbe:                     # до успеха liveness/readiness молчат
      httpGet: {path: /livez, port: http}
      failureThreshold: 20            # 20 × 2 с ≈ 40 с на старт
      periodSeconds: 2
    livenessProbe:                    # 3 неудачи подряд → рестарт контейнера
      httpGet: {path: /livez, port: http}
      periodSeconds: 10
      timeoutSeconds: 2
      failureThreshold: 3
    readinessProbe:                   # 2 неудачи → Pod убирают из endpoints
      httpGet: {path: /readyz, port: http}
      periodSeconds: 5
      failureThreshold: 2
    resources:
      requests: {cpu: 100m, memory: 64Mi}    # по этому scheduler ищет ноду
      limits: {cpu: 500m, memory: 256Mi}     # CPU — throttling, память — OOM
    securityContext:                  # это и требует restricted
      allowPrivilegeEscalation: false
      capabilities: {drop: ["ALL"]}
      readOnlyRootFilesystem: true    # запись только в /tmp → emptyDir
      runAsNonRoot: true
      runAsUser: 65532
```

Читаем: `image` закреплён по **digest** — sha256 не переедет, как тег; образ multi-arch, runtime возьмёт из индекса вариант под платформу ноды. `DB_HOST` нет — режим Memory. **requests** (100m = одна десятая CPU, 64Mi) — для размещения; **limits** — потолок: CPU режется throttling'ом, превышение 256Mi — OOM-завершение. Значения лабораторные; под нагрузку подбирают по измерениям. Для временных файлов ниже в файле `volumeMounts` + `volumes`: том `tmp` (emptyDir, 16Mi) на `/tmp` — данные не переживают удаление Pod.

Собес-ловушка: readiness ≠ liveness

Упал **readiness** (`/readyz`) — Pod исключают из обычной маршрутизации Service, но не перезапускают. Упал **liveness** (`/livez`) — kubelet перезапускает контейнер. У готовности есть конкретный потребитель: контроллер EndpointSlice публикует состояние endpoint'а, а Service использует его при выборе адресата. Оба probe ходят на порт по имени `http` — то самое 8080.

СОБЕС «Три probe с разными ролями: **startup** даёт время на старт (20 × 2 с) и до успеха глушит остальные; **liveness** /livez — рестарт после 3 неудач; **readiness** /readyz — Pod убирают из endpoints, не трогая. Образ закрепляю digest'ом, порт называю http и ссылаюсь по имени из probes и Service. requests — для scheduler, limits — потолок: CPU throttling, память OOM».

10 ПРИЛОЖЕНИЕ В КЛАСТЕРЕ · 3/5

Service ClusterIP и client-Pod: имя, порт 80 → http, EndpointSlice

Задача: Service даёт стабильное имя `devops-may-cry` и виртуальный адрес для подходящих Pod. Подходящие — по той же метке `app`. Клиент ходит на порт 80, `targetPort: http` ведёт на 8080 контейнера. ClusterIP виден только внутри кластера — поэтому рядом лежит диагностический Pod `client`.

LOCAL app/service.yaml · app/debug-client.yaml (фрагмент)

```yaml
kind: Service
metadata:
  name: devops-may-cry              # это имя резолвит cluster DNS
  namespace: traffic-lab
spec:
  type: ClusterIP                   # адрес только внутри кластера
  selector:
    app: devops-may-cry             # = template.metadata.labels в Deployment
  ports:
    - name: http
      port: 80                      # сюда идёт клиент
      targetPort: http              # → именованный порт контейнера = 8080
---
kind: Pod
metadata: {name: client, namespace: traffic-lab}
spec:
  automountServiceAccountToken: false
  securityContext: {runAsNonRoot: true, runAsUser: 1000, runAsGroup: 1000}
  containers:
    - name: netshoot                # сетевые утилиты: curl, dig, ...
      image: nicolaka/netshoot@sha256:<DIGEST>
      command: ["/bin/bash", "-lc", "sleep infinity"]   # живёт, ждёт exec
      securityContext:
        {allowPrivilegeEscalation: false, readOnlyRootFilesystem: true}
      volumeMounts: [{name: tmp, mountPath: /tmp}]      # rootfs read-only
```

Читаем цепочку: имя Service → порт 80 → endpoint выбранного Pod → порт 8080. Разойдутся метка и selector — у Service не будет endpoints. **EndpointSlice** отдельным YAML не описан: его создаёт контроллер — адреса и готовность выбранных Pod; Service выбирает по списку, запрос идёт напрямую в Pod. Client тоже непривилегированный, с `/tmp` на emptyDir — требование restricted.

i Ещё два файла: pdb.yaml и kustomization.yaml

PodDisruptionBudget с тем же selector и `minAvailable: 1`: при плановом выселении Pod (drain ноды на обслуживание) хотя бы одна реплика остаётся. **Kustomization** — вход Kustomize: `resources` перечисляет пять манифестов, и `kubectl apply -k app/` соберёт их и отправит в API одной командой.

СОБЕС «Service связан с Pod только меткой selector — разойдутся, и endpoints пусты. Клиент идёт на имя Service и port 80, а **targetPort по имени http** ведёт на 8080 контейнера. EndpointSlice я не пишу — его ведёт контроллер: адреса и ready выбранных Pod. ClusterIP снаружи не виден, поэтому проверяю из Pod внутри кластера».

10 ПРИЛОЖЕНИЕ В КЛАСТЕРЕ · 4/5

Запуск в kubespray: apply -k → rollout → Pod по нодам → EndpointSlice

Задача: отдать кластеру весь каталог одной командой, дождаться двух готовых реплик, посмотреть, где они оказались и что записал контроллер в EndpointSlice. Терминал 2, `KUBECONFIG` — учебная копия `recording.conf` из раздела 09.

CLUSTER контекст kubespray · терминал 2 · из 08_1.6_verify_cluster

```
cd ../../08_1.6_verify_cluster              # из клона Kubespray в лабораторию
kubectl --context kubespray apply -k app/   # -k: по kustomization.yaml

kubectl --context kubespray -n traffic-lab \
  rollout status deployment/devops-may-cry --timeout=300s

kubectl --context kubespray -n traffic-lab get pods -o wide   # + NODE и IP
```

ОЖИДАЕМЫЙ ВЫВОД

```
namespace/traffic-lab created
deployment.apps/devops-may-cry created   # + service, pdb, pod/client
deployment "devops-may-cry" successfully rolled out

NAME                              READY   STATUS    NODE    IP
devops-may-cry-xxxxxxxxxx-aaaaa   1/1     Running   node2   <POD_IP>
devops-may-cry-xxxxxxxxxx-bbbbb   1/1     Running   node3   <POD_IP>
client                            1/1     Running   node3   <POD_IP>
```

Читаем: при первом применении — пять строк `created`, при повторе `configured` / `unchanged`. `rollout status` держит терминал, пока доступных реплик не станет две; вышел по `--timeout` — смотри `describe` и события Pod. В NODE ждём разные ноды (kube_node у нас — node2 и node3); при ScheduleAnyway обе могут оказаться на одной — это не ошибка.

CLUSTER контекст kubespray · кого выбрал Service

```
kubectl --context kubespray -n traffic-lab get endpointslices \
  -l kubernetes.io/service-name=devops-may-cry -o yaml  # -l: по имени Service
```

ОЖИДАЕМЫЙ ВЫВОД

```
endpoints:
- addresses: ["<POD_IP>"]                      # IP реплики из get pods
  conditions: {ready: true, serving: true, terminating: false}
  nodeName: node2
- addresses: ["<POD_IP>"] …  nodeName: node3    # вторая реплика
ports:
- {name: http, port: 8080, protocol: TCP}      # тот самый targetPort http
```

СОБЕС «**apply -k** отправляет объекты в API — дальше кластер к ним приводят контроллеры. Сумме rollout status не верю, проверяю поимённо: **get pods -o wide** — READY 1/1 и NODE, затем **EndpointSlice** по метке kubernetes.io/service-name — адреса, ready: true и порт 8080. Когда «Service не отвечает», первым смотрю сюда».

Задача: последняя стрелка на доске — запрос. ClusterIP доступен только внутри, поэтому `curl` выполняем внутри Pod `client` через `kubectl exec`. Один запрос проверит четыре механизма разом.

POD контекст kubespray → exec в client · curl на имя Service

```
kubectl --context kubespray -n traffic-lab \
  wait --for=condition=Ready pod/client --timeout=120s

# DNS кластера + выбор endpoint'а + TCP до Pod + готовность — одним запросом
kubectl --context kubespray -n traffic-lab exec client -- \
  curl -fsS -w '\nHTTP %{http_code}\n' http://devops-may-cry/readyz

kubectl --context kubespray -n traffic-lab exec client -- \
  curl -fsS http://devops-may-cry/quote                    # цитата дежурного

kubectl --context kubespray -n traffic-lab exec client -- \
  curl -fsS 'http://devops-may-cry/order?on=dns-ghost'     # заявка
```

ОЖИДАЕМЫЙ ВЫВОД

```
pod/client condition met
<ответ /readyz>
HTTP 200                                  # ← приложение само сказало «готов»
{"pod":"devops-may-cry-xxxxxxxxxx-aaaaa","quote":"…","who":"…"}
{"order":1,"target":"dns-ghost","price":…,"status":"принят — дежурный выехал",
 "handled_by":"devops-may-cry-xxxxxxxxxx-bbbbb"}
```

Флаги: `exec client --` — после `--` идёт сама команда; `-fsS` — HTTP-ошибка считается ошибкой команды, без прогресс-бара, но с текстом ошибки; `-w '%{http_code}'` печатает код ответа отдельной строкой. **Читаем:** имя `devops-may-cry` разрешил cluster DNS → Service выбрал Ready-endpoint → TCP и HTTP дошли до контейнера на 8080 → приложение ответило 200. В `/quote` поле `pod` показывает, какая реплика ответила; в `/order` — `handled_by`.

i Короткое имя и режим Memory

Из того же namespace хватает `devops-may-cry`; полное имя — `devops-may-cry.traffic-lab.svc.cluster.local`. Номер заявки принадлежит конкретной реплике и сбрасывается с её перезапуском, истории нет — база не подключена. Pod `client` оставляем в namespace: этими же командами проверим кластер после обновления.

СОБЕС «Проверяю не «под поднялся», а весь путь: из Pod в том же кластере curl на имя Service — DNS разрешил, Service выбрал ready-endpoint, TCP дошёл до контейнера, приложение ответило 200 на /readyz. ClusterIP снаружи не виден, поэтому диагностический Pod netshoot держу в namespace постоянно».

10 ПРИЛОЖЕНИЕ В КЛАСТЕРЕ · 5/5

Те же манифесты в managed: другой контекст, ни строки правок

Задача: кластер собран по-другому, приложению это неважно. Тот же каталог `app/`, тот же kubectl, другой `--context`. В манифестах не меняем ничего.

CLUSTER контекст k8s-managed · тот же app/ · та же цепочка

```
kubectl --context k8s-managed apply -k app/
kubectl --context k8s-managed -n traffic-lab \
  rollout status deployment/devops-may-cry --timeout=300s
kubectl --context k8s-managed -n traffic-lab get pods -o wide
kubectl --context k8s-managed -n traffic-lab \
  wait --for=condition=Ready pod/client --timeout=120s
kubectl --context k8s-managed -n traffic-lab exec client -- \
  curl -fsS 'http://devops-may-cry/order?on=dns-ghost'
```

ОЖИДАЕМЫЙ ВЫВОД

```
namespace/traffic-lab created … pod/client created     # те же пять строк
deployment "devops-may-cry" successfully rolled out
NAME                              READY   STATUS    NODE
devops-may-cry-xxxxxxxxxx-ccccc   1/1     Running   <CLOUD_NODE_1> # от облака
devops-may-cry-xxxxxxxxxx-ddddd   1/1     Running   <CLOUD_NODE_2>
client                            1/1     Running   <CLOUD_NODE_1>
pod/client condition met
{"order":1,"target":"dns-ghost", …
 "handled_by":"devops-may-cry-xxxxxxxxxx-ccccc"}
```

Читаем: тот же образ, те же манифесты, другие ноды. В `handled_by` — имя реплики уже из managed-кластера. Приложение не знает, кто собрал кластер — Kubespray или облако.

ЧТО	KUBESPRAY	K8S-MANAGED
Манифесты, образ, ответ приложения	одинаковые — `apply -k app/`, два Pod, HTTP 200, заявка с `handled_by`	
Команда	одна и та же, отличается только `--context`	
Ноды в колонке NODE	`node2`, `node3` — наши VM из Terraform	worker-ноды с именами от облака
Кто собрал control plane	Kubespray на node1, node2	облако; в `get nodes` его нет

✓ **Один base — два overlay**

В `12_1.10_compare_clusters` тот же набор лежит как `base/` + `overlays/managed` и `overlays/self-managed`. Overlay добавляет на Deployment только метку `platform` — для нас, чтобы различать; приложению разница не нужна.

СОБЕС «Манифесты не зависят от способа установки кластера: тот же **apply -k** в k8s-managed — два Pod, тот же 200 и та же заявка. Отличаются только имена нод и то, кто отвечает за control plane. Kubernetes API — контракт; Kubespray и облако — две реализации под ним».

11 ДВА КЛАСТЕРА РЯДОМ · 1/2

Managed I Self-managed: семь строк границы ответственности

Задача: оба кластера работают — кладём их рядом. Один kubectl, два контекста: `k8s-managed` и `kubespray`. Идём по строкам таблицы и каждую из первых шести проверяем одной и той же командой в обоих контекстах; седьмая — итог.

#	MANAGED · K8S-MANAGED	SELF-MANAGED · KUBESPRAY
1 ноды control plane	в `get nodes` их нет — они за границей, у облака	`node1`, `node2` с ROLES control-plane, плюс `node3`
2 компоненты control plane	в kube-system нет kube-apiserver, scheduler, controller-manager — только coredns, kube-proxy, агенты облака	static Pod `kube-apiserver-node1`, `-controller-manager-node1`, `-scheduler-node1` и копии на node2
3 etcd и машины	машин control plane нет в списке VM: ни SSH, ни systemctl	etcd — наш systemd-сервис на node1–node3, заходим по SSH
4 доступ и отзыв	в kubeconfig — `exec yc`, токен от IAM на лету; отзыв — в IAM, централизованно	сертификат в файле, `kubernetes-admin`; отзыв — RBAC-binding и срок сертификата, руками
5 внешний IP для Service	LoadBalancer получает EXTERNAL-IP: балансировщик создаёт cloud-controller-manager	`<pending>` — балансировщика нет; MetalLB — следующая часть
6 версия и обновление	`k8s_version` + `release_channel` в Terraform, `auto_upgrade = false`	`kube_version` в group_vars, обновление — `upgrade-cluster.yml`
7 за что отвечаем мы	worker-ноды, сеть и правила, доступы, приложение	всё: ОС, runtime, etcd, control plane, сеть, обновления

CLUSTER строки 1–4 · парные команды · терминал 2, KUBECONFIG=recording.conf

```
# 1 · ноды: managed — только worker от облака; kubespray — node1, node2, node3
kubectl --context k8s-managed get nodes
kubectl --context kubespray get nodes
# 2 · kube-system: в managed нет kube-apiserver/scheduler/controller-manager
kubectl --context k8s-managed -n kube-system get pods
kubectl --context kubespray -n kube-system get pods
# 3 · машины: пять VM стенда + worker-ноды managed; control plane managed нет
yc compute instance list
"$KUBESPRAY_DIR/.venv/bin/ansible" \
  -i "$KUBESPRAY_DIR/inventory/video2/inventory.ini" \
  etcd --become -m command -a 'systemctl is-active etcd'   # active ×3
# 4 · доступ: users → exec yc vs client-certificate-data (DATA+OMITTED)
kubectl config view
kubectl --context k8s-managed auth whoami   # Username: ID аккаунта из IAM
kubectl --context kubespray auth whoami     # kubernetes-admin, cluster-admins
```

СОБЕС «Границу managed показываю командами, а не словами: в **get nodes** нет control-plane, в **kube-system** нет kube-apiserver, в списке VM облака нет их машин, доступ выдаёт IAM через exec. В своём кластере всё это моё: node1/node2, static Pod, etcd в systemd, сертификат admin в файле».

11 ДВА КЛАСТЕРА РЯДОМ · 2/2

Строки 5–6 практикой: LoadBalancer и версия кластера

Задача: объявить Service типа `LoadBalancer` — тот же selector, порт 80 наружу, `http` внутрь — и посмотреть, что сделают оба кластера; затем спросить версию у обоих API-серверов.

CLUSTER строки 5–6 · один манифест через stdin · оба контекста

```
kubectl --context kubespray -n traffic-lab apply -f - <<'EOF'
apiVersion: v1
kind: Service
metadata: {name: dmc-lb}
spec:
  type: LoadBalancer
  selector: {app: devops-may-cry}
  ports: [{name: http, port: 80, targetPort: http}]
EOF
kubectl --context kubespray -n traffic-lab get svc dmc-lb     # <pending>
# тот же heredoc с --context k8s-managed, затем:
kubectl --context k8s-managed -n traffic-lab get svc dmc-lb   # ждём адрес
curl -fsS http://<EXTERNAL_IP>/quote                          # с ноутбука
kubectl --context kubespray -n traffic-lab delete svc dmc-lb   # убираем
kubectl --context k8s-managed -n traffic-lab delete svc dmc-lb # LB исчезнет
# строка 6 · версия у обоих API-серверов
kubectl --context k8s-managed version   # Server Version: k8s_version + канал
kubectl --context kubespray version     # Server Version: v1.34.7 (group_vars)
```

ОЖИДАЕМЫЙ ВЫВОД

```
NAME     TYPE           CLUSTER-IP     EXTERNAL-IP     PORT(S)        AGE
dmc-lb   LoadBalancer   <CLUSTER_IP>   <pending>       80:3xxxx/TCP   # наш
dmc-lb   LoadBalancer   <CLUSTER_IP>   <EXTERNAL_IP>   80:3xxxx/TCP   # managed
```

Строка 5: в kubespray за LoadBalancer нет балансировщика — адрес выдавать некому, pending останется (MetalLB — следующая часть). В managed через минуту-две **cloud-controller-manager** создаёт сетевой балансировщик от имени сервисного аккаунта кластера (роль `cluster_lb` в `service-accounts.tf`) — приложение доступно снаружи без единой строки в кластере. Pending дольше 3 мин — `describe svc` → Events (раздел 12).
Строка 6: managed — `k8s_version` и `release_channel` в `private.auto.tfvars`, `auto_upgrade = false` в `kubernetes.tf`: версию меняем только явным `terraform apply`. Self-managed — `kube_version: 1.34.7` в `group_vars/k8s_cluster/k8s-cluster.yml`, обновление — playbook `upgrade-cluster.yml`.

i Строка 7 — за что отвечаем мы

Итог шести предыдущих. **Managed:** worker-ноды, сеть и правила, доступы, приложение. **Свой кластер:** всё — ОС, runtime, etcd, control plane, сеть, обновления. Приложению граница безразлична: одни манифесты работают в обоих.

✓ Строки 1–6 показываем командами в двух контекстах

Один kubectl, два `--context`, команды парами — граница видна в самих выводах.

СОБЕС «Граница ответственности — то, чего я не вижу в выводе: в managed нет control-plane нод, их Pod и VM, доступ выдаёт IAM, адрес LoadBalancer — cloud-controller-manager, версию держит канал в Terraform. В своём кластере всё это моё — включая **‹pending›** без MetalLB и upgrade-cluster.yml руками».

Terraform: provider не авторизован · Saved plan is stale

Симптом А: новый терминал, `terraform plan` — и вместо плана `"folder_id" is required` или ошибка авторизации. Код не менялся. Причина в окружении: provider yandex читает `YC_TOKEN`, `YC_CLOUD_ID`, `YC_FOLDER_ID` из переменных shell, а новая сессия их не наследует.

LOCAL шаг 1 — есть ли переменные в ЭТОЙ сессии

```
env | grep '^YC_'                          # пусто = provider слепой
terraform plan -var-file=env/video.tfvars
```

ВЫВОД В СВЕЖЕМ ТЕРМИНАЛЕ

```
Error: "folder_id" is required
  with provider["registry.terraform.io/yandex-cloud/yandex"]
```

Читаем: в `providers.tf` токена нет намеренно — так секрет не попадает в git. Плата: каждая новая сессия shell начинается с пустого окружения. Это не баг Terraform и не повод вписывать токен в `.tf`.

LOCAL фикс — заново выдать доступ сессии (IAM-токен живёт до 12 ч)

```
export YC_TOKEN="$(yc iam create-token)"
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
test -n "$YC_TOKEN"; echo $?          # 0 = непустой; сам токен не печатаем
terraform plan -var-file=env/video.tfvars    # теперь план строится
```

Симптом Б: `terraform apply video2.tfplan` → `Saved plan is stale`

Сохранённый план — снимок «код + state + реальность» на момент `plan -out`. Любое изменение state после него (другой apply, правка tfvars, обновление outputs) делает файл недействительным: Terraform отказывается применять устаревший прогноз.

LOCAL фикс — план заново, применить свежий файл

```
terraform apply video2.tfplan
# Error: Saved plan is stale — state changed since the plan was created
terraform plan -var-file=env/video.tfvars -out=video2.tfplan  # читаем diff!
terraform apply video2.tfplan
```

Собес-ловушка: «положу токен в tfvars, чтобы не экспортировать»

tfvars коммитят, state и план читают коллеги — токен уедет в git. Правильно: credentials через окружение или workload identity, в новой сессии выдаются заново. Старый `.tfplan` не «докручивают» — строят новый и читают целиком.

СОБЕС «Provider не авторизован в новом терминале — это переменные окружения, а не код: проверяю `env | grep YC_`, заново экспортирую токен и ID. `Saved plan is stale` значит, что state изменился после сохранения плана — переделываю `plan -out` и применяю свежий файл, старый не чиню».

TROUBLESHOOTING · 2/5

Квоты каталога: сеть не создаётся, node group висит в Provisioning

Симптом А: `apply` пяти VM падает на сети: `quota vpc.networks.count exceeded`. **Симптом Б:** в managed control plane поднялся за минуты, а `apply` час ждёт node group и выходит по `context deadline exceeded`. Код ни при чём — квоты каталога.

LOCAL А — кто занял квоту сетей и как освободить

```
yc vpc network list                    # в каталоге уже лежит сеть default
yc vpc subnet list                     # её подсети удаляются первыми
yc vpc subnet delete <SUBNET_NAME> && yc vpc network delete default
```

Читаем: новый каталог приходит с сетью `default` — она тоже считается в `vpc.networks.count`. Не используешь — снеси; нужна — запроси квоту (как в Б). Потом снова `plan` → `apply`.

LOCAL Б — почему вторая worker-нода не создаётся

```
yc managed-kubernetes node-group list-nodes --name k8s-managed-workers
yc quota-manager quota-limit get --service compute \
  --quota-id compute.ssdDisks.size --resource-type resource-manager.folder \
  --resource-id <FOLDER_ID>
```

ВЫВОД (ГРУППА В NODE-GROUP LIST — PROVISIONING, SIZE 2)

```
node-b   PROVISIONING / CREATING_INSTANCE   # node-a уже READY
         [RESOURCE_EXHAUSTED] The limit on total size of network-ssd disks
limit: 214748364800   usage: 193273528320   # 200 GiB, занято 180
```

Читаем: причина — у конкретного узла в `list-nodes`. Считаем: пять VM стенда — 150 GiB SSD, первая worker — 30, итого 180; второй диск на 30 в 200 не влезает. Control plane облако подняло само; диски нод — наша сторона границы.

LOCAL фикс Б — квота с запасом (236223201280 байт = 220 GiB) или ужать группу

```
yc quota-manager quota-request create --resource-id <FOLDER_ID> \
  --resource-type resource-manager.folder \
  --desired-limit quota-id=compute.ssdDisks.size,value=236223201280
# запасной путь: node_count = 1 в private.auto.tfvars → plan → apply
```

✓ **Фикс по горячим следам**

Таймаут `apply` — лимит ожидания провайдера, не ответ облака. Второй `apply` поверх не запускай: `quota-request list` → APPROVED (мелкие заявки одобряются за минуты), группа доделает ноду сама, а `terraform plan` даст `No changes` — группа уже в state.

СОБЕС «„Облако сделает само“ тоже упирается в квоты каталога: сеть `default` считается в `vpc.networks.count`, диски worker-нод — в `compute.ssdDisks.size`. Диагностирую `yc … list-nodes` и `quota-limit get`, прошу квоту с запасом через `quota-request create`, а не жду таймаут Terraform».

SSH и kubectl зависли: allowlist /32 устарел · preflight остановился

Симптом А: посреди работы `ssh ubuntu@<NODE1_IP>` и `kubectl get nodes` висят до `timeout`. Ноды RUNNING, ключ тот же. Security group пускает 22 и 6443 только с `<YOUR_IP>/32`, а публичный адрес ноутбука сменился (провайдер, VPN).

LOCAL A — сверить текущий адрес с allowlist (в managed: api и nodeport cidr)

```
curl -4 -fsS https://api.ipify.org; echo     # кем нас видит облако сейчас
grep allowed_ 04_1.3_five_vm_terraform/terraform/env/video.tfvars
grep allowed_ 05_1.4_managed_cluster/terraform/private.auto.tfvars
```

```
ВЫВОД
<NEW_IP>
allowed_ssh_cidr = "<OLD_IP>/32"   # ≠ текущему → SG режет пакеты
```

LOCAL фикс А — новый CIDR в tfvars → plan → apply: меняются только правила SG

```
# tfvars: allowed_ssh_cidr = "<NEW_IP>/32" (и allowed_api/nodeport_cidr)
terraform plan -var-file=env/video.tfvars -out=video2.tfplan
# ждём: yandex_vpc_security_group.this updated in-place, 1 to change
terraform apply video2.tfplan
```

Читаем: план обязан менять только security group. Появились `replace` / `destroy` у VM — стоп и разбираем, откуда (например, обновился образ в `data`).

Симптом Б: preflight упал на `overlay` / `br_netfilter` / `ip_forward = 0`

LOCAL 06_1.4bis_node_preflight — assert только проверяет, prepare-network чинит

```
AP="$KUBESPRAY_DIR/.venv/bin/ansible-playbook"
$AP playbooks/preflight.yml
# fatal: [node1]: FAILED! => assert regex('(?m)^overlay\\s') failed
$AP playbooks/prepare-network.yml --list-hosts   # только node1, node2, node3
$AP playbooks/prepare-network.yml                # modules-load.d + sysctl.d
$AP playbooks/preflight.yml                      # → failed=0 на трёх нодах
```

✓ **Ожидаемая остановка, не авария**

Preflight читает и не чинит. Kubespray сам загрузит `overlay` (ExecStartPre containerd), `br_netfilter` (роль kubernetes/node) и включит `ip_forward` (preinstall) — но готовим заранее, чтобы требования к ноде были видны и независимая проверка стала зелёной.

СОБЕС «Allowlist по /32 — правильная защита SSH и API, но её надо уметь обновлять: сверяю `curl -4 api.ipify.org` с tfvars, правлю CIDR, plan показывает одно изменение у SG. Preflight с assert на модули и forwarding — ожидаемый стоп: применяю prepare-network и повторяю проверку до `failed=0`».

Kubespray: download падает на одной ноде — TLS handshake timeout

Симптом: `cluster.yml` на этапе download валится на node1: `Failed to download … dl.k8s.io / github.com … TLS handshake timeout`, проходит одна попытка из многих. На node2 та же задача — `ok`. Inventory и group_vars ни при чём: конкретная VM плохо ходит к CDN.

NODE1 шаг 1 — воспроизвести мимо Kubespray: curl и pull руками

```
curl -sSI --max-time 20 \
  https://dl.k8s.io/release/v1.34.7/bin/linux/amd64/kubectl | head -1
sudo nerdctl pull registry.k8s.io/pause:3.10     # образ через containerd
```

ВЫВОД НА БОЛЬНОЙ НОДЕ (НА NODE2 — HTTP/2 200)

```
curl: (28) Operation timed out after 20000 milliseconds
error: … net/http: TLS handshake timeout        # pull тоже не проходит
```

Читаем: на node2 те же команды проходят — значит, не Kubespray и не DNS кластера (его ещё нет), а сетевой путь этой VM к CDN. Kubespray скачивает бинарники и образы **на каждую ноду**; выход нод наружу — часть preflight, которую наш assert не проверял.

ЧТО ДЕЛАТЬ	КАК	ГДЕ ЗАДАВАТЬ
Перезапустить playbook	та же команда: кэш в `/tmp/releases` на ноде переиспользуется, checksum сверяется	—
Зеркало реестра образов	`containerd_registries_mirrors`: `prefix` + `server` + `mirrors: host`	свой `group_vars` или `-e`, не clone
Бинарники с другого источника	`files_repo` / `*_download_url`	там же

LOCAL фикс — зеркало в СВОИХ group_vars, затем повторный запуск

```
# inventory/video2/group_vars/all/containerd.yml
containerd_registries_mirrors:
  - prefix: registry.k8s.io
    server: https://<MIRROR_HOST>
    mirrors:
      - host: https://<MIRROR_HOST>
        capabilities: ["pull", "resolve"]
.venv/bin/ansible-playbook -i inventory/video2/inventory.ini \
  --become cluster.yml                          # повторный запуск
```

СОБЕС «Download падает на одной ноде, на другой проходит — сеть VM, не установщик: проверяю `curl --max-time 20` и `nerdctl pull` руками на ноде. Повтор дотянет из кэша `/tmp/releases`; упорная проблема — зеркала через `containerd_registries_mirrors` и `files_repo` в своих group_vars, клон не правлю».

TROUBLESHOOTING · 5/5

LoadBalancer в managed висит в ‹pending›: имя порта и права SA

Симптом: в `kubespray` `<pending>` — норма: за Service там нет балансировщика (MetalLB — следующая часть). Но в `k8s-managed` тот же `dmc-lb` через три минуты без EXTERNAL-IP. Балансировщик заказывает cloud-controller-manager от имени SA кластера — идём в Events.

K8S-MANAGED шаг 1 — Events сервиса: на записи было две причины подряд

```
kubectl --context k8s-managed -n traffic-lab get svc dmc-lb
kubectl --context k8s-managed -n traffic-lab describe svc dmc-lb | tail -6
```

ВЫВОД — СНАЧАЛА (А), ПОСЛЕ ЕЁ ФИКСА (Б)

```
dmc-lb   LoadBalancer   10.96.x.x   <pending>   80:3xxxx/TCP
Warning  SyncLoadBalancerFailed  listener name "80-8080" is invalid:
                                 must start with a letter          # (а)
Warning  SyncLoadBalancerFailed  PermissionDenied: service account
                                 lacks role load-balancer.admin    # (б)
```

Читаем: (а) имя listener'а облако выводит из имени порта Service; порт без `name` получает `80-8080` — с цифры, облако отвергает. (б) `k8s.clusters.agent` даёт управлять кластером, а создавать NLB — отдельная роль `load-balancer.admin`, которой у SA кластера не было.

LOCAL фикс (а) — имя порту · фикс (б) — роль SA через Terraform, не в консоли

```
# (а) в Service: ports: - name: http   port: 80   targetPort: http
kubectl --context k8s-managed -n traffic-lab apply -f dmc-lb.yaml

# (б) 05_1.4_managed_cluster/terraform/service-accounts.tf — ещё один member
resource "yandex_resourcemanager_folder_iam_member" "cluster_lb_admin" {
  folder_id = data.yandex_client_config.client.folder_id
  role      = "load-balancer.admin"
  member    = "serviceAccount:${yandex_iam_service_account.cluster.id}"
}
terraform plan -out=managed.tfplan && terraform apply managed.tfplan
kubectl --context k8s-managed -n traffic-lab get svc dmc-lb -w  # EXTERNAL-IP
curl -fsS http://<EXTERNAL_IP>/quote                            # с ноутбука
```

Собес-ловушка: «дам роль в консоли, так быстрее»

Сработает — и на следующем `terraform plan` вылезет drift: binding, сделанный руками, коду неизвестен. Права SA — часть кода кластера: правим `service-accounts.tf`, apply, повторный plan в ноль. IAM-роли «прожимаются» не мгновенно — `depends_on` задаёт порядок, но не распространение прав.

СОБЕС «LoadBalancer в managed — это cloud-controller-manager, который создаёт балансировщик от имени сервисного аккаунта кластера. `<pending>` читаю через `describe svc` → Events: у нас было имя порта с цифры и нехватка роли `load-balancer.admin`. Права SA — граница managed, и правлю их в Terraform, а не в консоли».

Terraform: цикл, state, drift и входы

1 Что делает `terraform init` ? Чем замыкается цикл init → validate → plan → apply?

ЛОВУШКА

«init создаёт ресурсы» или «init проверил доступ к облаку». Он ставит зависимости; вызовы API начинаются с plan.

ОТВЕТ

init готовит каталог: backend, providers из `required_providers`, `.terraform` и lock-файл. **validate** — синтаксис и ссылки без сети. **plan** сравнивает код, state и реальность; **apply** строит тот же план и после «yes» приводит реальность к коду, записывая state. Замыкает цикл повторный plan: `No changes`.

JR / MD / SR

jr: пять команд по порядку · md: `-upgrade` осознанно, lock-файл в git · sr: `plan -out` + apply файла; `Saved plan is stale` = state изменился после снимка.

2 Что такое state и почему в команде его не хранят файлом в git?

ЛОВУШКА

«state — бэкап инфраструктуры». Удалил state — объекты живут, но без хозяина.

ОТВЕТ

State — память Terraform: адрес в коде ↔ реальный объект и его атрибуты (`state list`, `state show`). Без него plan не отличит «создать» от «уже есть». В git опасен: два apply параллельно перетирают друг друга, плюс в state лежат чувствительные атрибуты. Стандарт — remote backend с блокировкой.

3 Что такое drift и как с ним работаешь?

ЛОВУШКА

«drift — любой diff в plan». Правка кода — обычный workflow; drift — когда объект изменили мимо Terraform.

ОТВЕТ

Объект поправили руками (консоль, ssh), код и state не знают. Ловится plan на чистом коде: дописали строку в `build/journal.txt` — plan ответил `will be created, 1 to add`. Развилка: правка мусор → осознанный apply вернёт описанное; правка нужна → сначала в код, потом plan в ноль. Никогда: удалять state или apply вслепую — снесёшь чей-то хотфикс.

JR / MD / SR

md: у provider local — пересоздание по checksum, у облачных — update in-place / replace · sr: plan по расписанию в CI, ручные правки закрыты правами.

4 variable, local, output — что где? Почему пять VM через `for_each` , а не `count` ?

ЛОВУШКА

«local — та же переменная, только внутри». local не передаётся через `-var-file`; это вычисление, не вход.

ОТВЕТ

variable — вход: tfvars через `-var-file`, `validation`, без default у `allowed_ssh_cidr`. **local** — вычисление: карта `nodes` с ролями, `api_endpoint_nodes` через for/if/try. **output** — результат наружу: `terraform output -raw kubespray_inventory > inventory.ini`. Один resource с `for_each = var.nodes`: ключ `node2` стабилен, node4 не сдвигает соседей; под count удаление из середины пересоздаёт невиновных. `dynamic` — вложенный блок по условию (`fixed_scale` / `auto_scale`).

Ansible: идемпотентность, модули, handler, precedence

5 За счёт чего Ansible идемпотентен — и где это заканчивается?

ЛОВУШКА

«Ansible гарантирует идемпотентность любого playbook» — или наоборот «вообще не поддерживает».

ОТВЕТ

Сравнение состояния живёт **в модулях**: `apt` спрашивает dpkg, `file` смотрит существование/тип/права, `template` рендерит и сравнивает с диском. Совпало — `ok`: второй прогон `site.yml` дал `changed=0`. Это свойство модуля и параметров: `service state: started` идемпотентен, `restarted` — перезапуск каждый раз; `shell` с `>>` дописывает строку бесконечно.

JR / MD / SR

jr: «модуль сравнивает» · md: `lineinfile`, `creates`/`removes`, честный `changed_when` · sr: `changed_when: false` «для красоты» = выключенная сигнализация.

6 command vs shell vs raw — когда что?

ЛОВУШКА

«command и shell — одно и то же». В command нет оболочки: pipe и `>>` не сработают.

ОТВЕТ

command запускает бинарь напрямую — pipes/redirects не интерпретируются. **shell** гонит строку через `/bin/sh`: `ss -tlnp | grep 8080` законен, вместе с сюрпризами экранирования. **raw** шлёт строку по SSH без Python на той стороне — bootstrap (`apt-get install -y python3`). Все трое не знают желаемого состояния: `whoami` через `--become` отдал CHANGED, ничего не изменив.

7 Что такое handler и когда он срабатывает?

ЛОВУШКА

«handler запускается при каждом прогоне задачи» или «notify срабатывает и при ok».

ОТВЕТ

Handler — задача, которую task ставит в очередь через `notify` **только при changed**. Выполняется в конце play, очередь дедуплицируется: пять изменённых конфигов с notify на «Restart app» — один рестарт. Наш handler сработал на первом прогоне и на restore после drift, молчал на обоих повторах. Нужен раньше конца play — `meta: flush_handlers`.

JR / MD / SR

md: restart живёт в handler, не в tasks — иначе рестарт каждый прогон · sr: `--check --diff` предсказывает правку файла, но command в check пропускается; RUNNING HANDLER в check ≠ изменённый файл.

8 Откуда взялось значение переменной: defaults, group_vars, host_vars?

ЛОВУШКА

«любой более конкретный файл всегда побеждает». Это цепочка inventory-уровней, а не универсальный закон.

ОТВЕТ

В лаборатории: `defaults` роли (самые слабые) → `group_vars/all` → `group_vars/night_office` → `host_vars/localhost`. Итог виден в файле: `gear` из host_vars, `dispatcher_name` из группы, `office_path` общий — `cat /tmp/devops-may-cry/dispatcher.conf`. Не то значение — идёшь уровень за уровнем, а не правишь шаблон вслепую. `vars/` роли выше inventory, `-e` выше всего.

Kubespray и установка: inventory, стадии, static Pod, etcd

9 Что такое Kubespray и чем он отличается от kubeadm?

ЛОВУШКА

«Kubespray — свой дистрибутив Kubernetes» / «замена kubeadm». Он ставит обычный Kubernetes и вызывает kubeadm внутри.

ОТВЕТ

Kubespray — Ansible-проект Kubernetes SIGs: inventory (кто control plane / etcd / worker) + group_vars (`kube_version`, containerd, Calico, CIDR) + готовые роли; корневой `cluster.yml` — обёртка с `import_playbook`. kubeadm — инициализация и join; Kubespray добавляет подготовку ОС, runtime, etcd, сеть, addons. Версию закрепляем tag `v2.31.0` и отдельным `.venv`: ansible-core 2.18.x, ниже 2.19.

JR / MD / SR

jr: «Ansible ставит k8s» · md: `git clone --branch v2.31.0 --depth 1`, `git describe --tags --exact-match` · sr: матрица версий release — 1.34.7 и 1.35.4 внутри одного tag, отсюда план обновления.

10 В inventory у ноды `ansible_host` и `ip` / `access_ip` — зачем два адреса?

ЛОВУШКА

«пиши везде внешний — по нему же ssh ходит». В облаке внешний адрес — NAT, на интерфейсе VM его нет.

ОТВЕТ

`ansible_host` — куда Ansible заходит по SSH (внешний). `ip` / `access_ip` — внутренний адрес, который займут etcd и kubelet и по которому ноды ищут друг друга. Внешний во внутренних полях — etcd пытается забиндить адрес, которого нет, и установка ложится на первом сервисе. Адреса не переписываем руками: `terraform output -raw kubespray_inventory`, проверка `ansible-inventory --graph`.

11 Что делает `cluster.yml` по этапам и почему kubelet ставится раньше control plane?

ЛОВУШКА

«api-server ставится как Deployment через kubectl». Через какой API, если его ещё нет?

ОТВЕТ

Семь plays: preinstall (ОС, download на **каждую** ноду) → containerd → etcd → kubelet на всех нодах → control plane (`kube_control_plane`) → kubeadm join + Calico → addons (CoreDNS, nodelocaldns). kubelet раньше: kubeadm кладёт манифесты в `/etc/kubernetes/manifests`, и именно kubelet поднимает api-server, scheduler, controller-manager как **static Pod** (в API — mirror Pod). До CNI ноды в кластере, но NotReady.

JR / MD / SR

md: preflight до запуска — `ansible all -m ping` (SSH+Python), `--become -a whoami`, swap off, overlay/br_netfilter, ip_forward=1 · sr: Kubespray сам чинит модули и forwarding, но egress нод к CDN не проверяет — добавь в preflight.

12 Почему etcd три и почему его нет в `kube-system` ?

ЛОВУШКА

«4 ноды etcd надёжнее 3». Кворум N/2+1: у четырёх — три голоса, терпит тот же один отказ.

ОТВЕТ

Три участника — большинство два, один отказ переживаем; четвёртый ничего не добавляет, для двух отказов нужно пять. У нас `etcd_deployment_type: host` — бинарь под systemd на node1–3, поэтому Pod etcd в kube-system нет: `ansible etcd --become -a 'systemctl is-active etcd'`. Хранит данные API, не заявки приложения.

Managed vs self-managed и доступ к кластеру

13 Где проходит граница ответственности в managed? Плюсы и минусы.

ЛОВУШКА

«managed = облако отвечает за всё». Ниже границы всё наше.

ОТВЕТ

Control plane и etcd скрыты за API: `get nodes` показывает только worker, в kube-system нет apiserver/scheduler/controller-manager, их VM нет в `yc compute instance list`. Наше: worker-группы (размер, диски, квоты), приложения, requests/limits, probes, RBAC, данные, мониторинг, стоимость. Плюс — меньше дежурства по control plane; минус — версии и флаги в темпе провайдера, зависимость от квот и ролей.

JR / MD / SR

md: `auto_repair` worker-групп у сервиса, `auto_upgrade` выключили · sr: одни манифесты работают в обоих кластерах — приложению граница безразлична, разница в дежурстве.

14 Чем kubeconfig managed отличается от self-managed? Что при утечке?

ЛОВУШКА

«kubeconfig — просто адрес кластера». В `users` лежит либо exec-плагин, либо сам ключ.

ОТВЕТ

kubeconfig = **clusters** (сервер + CA) + **users** + **contexts**. В managed в users — `exec`: kubectl зовёт `yc` и получает IAM-токен на лету (до 12 ч), в файле токена нет — утечка без облачной учётки бесполезна. В self-managed — `client-certificate-data` и `client-key-data`: утёк файл — утёк доступ, списка отзыва у Kubernetes нет. Смотрю `kubectl config view` (DATA+OMITTED) и `kubectl auth whoami`.

JR / MD / SR

sr: для команды в своём кластере — OIDC (`oidc-issuer-url`), binding на группу из токена; для одного админа хватает сертификата со сроком.

15 Новому администратору нужен доступ. Как выдашь и как отзовёшь?

ЛОВУШКА

«перешлю admin.conf». Это cluster-admin без имени: не привязать к человеку и не отозвать.

ОТВЕТ

Личность, права, файл. Свой кластер: коллега делает ключ и CSR (CN = имя), я — `CertificateSigningRequest` со сроком и `kubectl certificate approve`; права — (Cluster)RoleBinding на `admin` / `cluster-admin`; файл — адрес API, CA, его сертификат; проверка `auth can-i … --as=ivan`. Отзыв — удалить binding. Managed: личность уже в IAM — `yc … add-access-binding`, файл сам через `get-credentials`; отзыв — `remove-access-binding`.

16 Security group пускает только `/32`. Зачем и что делать, когда IP сменился?

ЛОВУШКА

«на время открою 0.0.0.0/0, потом закрою». «Потом» не наступает; `validation` в variables.tf такое значение отвергает.

ОТВЕТ

`allowed_ssh_cidr` — адрес, с которым мы приходим в облако (`curl -4 https://api.ipify.org`), не IP ноды и не 192.168.x. `/32` = ровно один адрес: TCP 22 и 6443 только оттуда; egress `0.0.0.0/0` — исходящий, другое правило. IP сменился (VPN, провайдер) — ssh и kubectl висят до timeout: обновляю tfvars, `plan` показывает `1 to change` только у SG, apply.

Манифесты и трафик внутри кластера

17 Что дают метки `pod-security` на namespace и зачем `securityContext` ?

ЛОВУШКА

«это PodSecurityPolicy / флаг api-server». PSP удалён; PSA настраивают метками namespace.

ОТВЕТ

Метки `enforce` / `warn` / `audit` = restricted: не пустить, предупредить в kubectl, записать в audit. Restricted требует `runAsNonRoot`, `allowPrivilegeEscalation: false`, `drop: ["ALL"]`, seccomp `RuntimeDefault`; у нас ещё `readOnlyRootFilesystem` + emptyDir на /tmp. Без securityContext apply откажет.

JR / MD / SR

md: уровни privileged / baseline / restricted · sr: сначала warn+audit, enforce после; `automountServiceAccountToken: false`, если API поду не нужен.

18 `replicas: 2` — реплики точно окажутся на разных нодах?

ЛОВУШКА

«да, scheduler разнесёт». Без ограничений обе могут сесть на одну ноду.

ОТВЕТ

Нет. `topologySpreadConstraints`: `maxSkew: 1` по `kubernetes.io/hostname` — разница между нодами ≤ 1. `ScheduleAnyway` — мягко, обе могут сесть на node2 (смотри `-o wide`); `DoNotSchedule` — жёстко, под в Pending. PDB `minAvailable: 1` — про drain, не про краш.

19 Как связаны selector Deployment, labels Pod и selector Service? Что в EndpointSlice?

ЛОВУШКА

«Service находит поды по имени Deployment». Только по labels.

ОТВЕТ

`selector.matchLabels` Deployment = `template.metadata.labels` (`app: devops-may-cry`) — контроллер ↔ его Pod. У Service та же метка в `selector`; контроллер EndpointSlice пишет адреса Pod и `conditions.ready`. Разошлись метки — endpoints пусто, curl висит. `targetPort: http` → 8080.

20 Три probe — и путь `curl http://devops-may-cry/readyz` из client-пода.

ЛОВУШКА

«readiness провалилась → под перезапустят». Нет: только вывод из endpoints; рестартит liveness.

ОТВЕТ

startup (`/livez`, 20×2 с) держит остальные на старте; **liveness** — 3 провала → рестарт; **readiness** `/readyz` с Ping хранилища — провал → endpoint not ready. Путь: CoreDNS → ClusterIP:80 → правила kube-proxy → ready-endpoint из EndpointSlice → Pod:8080 → `HTTP 200`: DNS, Service, TCP и готовность одним запросом.

JR / MD / SR

sr: `type: LoadBalancer` в kubespray — `<pending>`, реализации нет (MetalLB — следующая часть); в managed NLB создаёт cloud-controller-manager от имени SA кластера.

i Как отвечать на собесе

Минимальный корректный ответ → «рассказать подробнее?» → углубляй по грейду. Опирайся на сделанное руками: «plan дал 1 to add», «второй прогон — changed=0», «в managed get nodes без control-plane».

8 заданий — каждое с промтом для ИИ

Тетрадь работает, только если повторяешь руками. 8 заданий на твоём стенде; к каждому — промт для нейронки: проси **объяснить → дать команды → задать тебе вопрос**.

№1 · Hello Terraform: drift руками

После apply допиши строку в `build/journal.txt` мимо Terraform → plan «1 to add» → apply → контрольный plan «No changes».

ПРОМТ

```
Hello Terraform, provider local: я дописал строку в управляемый файл руками. Почему plan показывает «will be created», хотя файл на диске есть? Два пути (мусор / нужна) и команды доказать, что drift закрыт.
```

№2 · Hello Ansible: host_vars и precedence

Добавь `dispatcher_name` в host_vars/localhost.yml → site.yml → cat dispatcher.conf: host_vars бьёт group_vars. Потом `precedence.yml`: play vars бьют host_vars.

ПРОМТ

```
gear задан на пяти уровнях: defaults роли, group_vars/all, group_vars/night_office, host_vars, vars в play. Объясни порядок precedence и дай команду увидеть итоговое значение без запуска playbook. Спроси, где место секретам.
```

№3 · Неидемпотентная задача → модуль

`shell` с >> (BAD из non_idempotent_examples.yml): два прогона — файл растёт, всегда changed. Перепиши через `lineinfile` или command + creates → changed=0.

ПРОМТ

```
Задача: shell: echo 'debug=true' >> /etc/app.conf. Почему она всегда changed и чем опасна? Перепиши через lineinfile, command с creates и changed_when: что сравнивает каждый?
```

№4 · Terraform стенда: шестая VM — только plan

Добавь `node4` (worker) в local.nodes в locals.tf. plan: «1 to add», ноль изменений у пяти существующих. Не применяй — деньги и квота SSD.

ПРОМТ

```
Я добавил шестую VM в карту local.nodes; модуль создаёт VM через for_each по ней. Объясни по plan, почему пять существующих не пересоздаются и что было бы с count. Как увидеть группы inventory для node4, не применяя план?
```

Задания 5–8 · inventory, preflight, кластер

№5 · Inventory из outputs: меняем роль node3

Убери у node3 роль worker в locals.tf: plan меняет только outputs. Примени, `output -raw kubespray_inventory` — node3 ушёл из [kube_node]. Верни как было.

ПРОМТ

```
Inventory для Kubespray генерирует output Terraform из карты машин. Я поменял роль node3, а terraform output показал старый inventory. Почему output читает state, а не код? Что сделать между правкой и экспортом?
```

№6 · Preflight: ломаем ip_forward и ловим

До cluster.yml (или на отдельной VM) выставь `net.ipv4.ip_forward=0` → preflight.yml: красный assert → prepare-network.yml → preflight ok. На живой ноде не ломай.

ПРОМТ

```
Preflight упал на assert net.ipv4.ip_forward == 1 на node2. Объясни, зачем ноде forwarding, overlay и br_netfilter, и чем sysctl «сейчас» отличается от sysctl.d «после перезагрузки». Дай команды проверить оба состояния.
```

№7 · Манифесты: сломай selector → пустой EndpointSlice

В app/service.yaml сломай selector (`app: devops-may-cry-x`) → apply -k: EndpointSlice пуст, curl из client падает. Верни, проверь /readyz. В Deployment selector immutable.

ПРОМТ

```
Deployment и Service, selector app: devops-may-cry. Как Service находит поды по labels, кто заполняет EndpointSlice? Сломай selector Service, покажи пустые endpoints, почини.
```

№8 · Два контекста одной командой

`for c in kubespray k8s-managed; do kubectl --context $c -n kube-system get pods; done` — где apiserver, scheduler, controller-manager и почему в managed их нет.

ПРОМТ

```
В kube-system у kubespray есть kube-apiserver-node1, у k8s-managed — нет. Где живёт control plane в managed и почему get nodes его не показывает? Ещё 3 команды на границу.
```

✓ Прошёл все 8 — ты поднимал, ломал, чинил

На собесе это слышно: «ломал файл, check предсказал правку, повтор дал changed=0».

Terraform и доступ к облаку

Доступ к облаку — переменные окружения YC_*

LOCAL yc → окружение → provider yandex (в .tf токена нет)

```
yc config profile list                      # активный профиль; нет — yc init
export YC_TOKEN="$(yc iam create-token)"    # IAM-токен ~12 ч, не печатать
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
test -n "$YC_TOKEN"; echo $?                # 0 = непустой (срок не проверяет)
yc resource-manager folder get --id "$YC_FOLDER_ID"   # доступ к каталогу есть
printf 'cloud=%s folder=%s\n' "$YC_CLOUD_ID" "$YC_FOLDER_ID" # без токена
```

Рабочий цикл

LOCAL hello: env/demo.tfvars · стенд: env/video.tfvars · managed: *.auto.tfvars

```
terraform init                              # provider + lock-файл, ресурсов 0
terraform validate                          # синтаксис и связи, без облака
terraform plan -var-file=env/video.tfvars -out=video2.tfplan  # план в файл
terraform apply video2.tfplan               # ровно этот план, без yes
terraform plan -out=managed.tfplan          # managed: из *.auto.tfvars
terraform apply -var-file=env/demo.tfvars   # hello: интерактивно, спросит yes
terraform plan -var-file=env/demo.tfvars    # контроль: No changes
terraform fmt -check -recursive            # из README стенда, в кадре не было
```

Saved plan is stale: state изменился после сохранения плана — строй plan заново, старый tfplan не применяй. **No changes** = код, state и реальность совпадают.

State, outputs, уборка

LOCAL state — что Terraform помнит; output читает state, а не код

```
terraform state list                        # адреса управляемых ресурсов
terraform state show 'module.linux_vm.yandex_compute_instance.this["node1"]'
terraform output                            # все outputs (после apply)
terraform output nodes                      # один output, для человека
terraform output -raw kubespray_inventory   # сырой текст, без кавычек
terraform output -raw kubespray_inventory \
  > ../../07_1.5_kubespray_full/inventory/video2/inventory.ini
terraform output -raw kubespray_all_yml \
  > ../../07_1.5_kubespray_full/inventory/video2/group_vars/all/all.yml
terraform destroy -var-file=env/video.tfvars # только закончив практику; читай план!
```

Ansible, Kubespray и yc

Ansible — hello-лаба и preflight

LOCAL из корня лабы: ansible.cfg сам находит inventory и roles

```
ansible --version                          # версия + какой ansible.cfg
ansible-inventory --graph                  # группы; по SSH не ходит
ansible-playbook playbooks/site.yml --syntax-check
ansible-playbook playbooks/site.yml        # 1-й changed>0, 2-й changed=0
ansible-playbook playbooks/site.yml --check --diff   # предсказать, не меняя
ansible-playbook playbooks/site.yml --diff  # применить, показать before/after
ansible-playbook playbooks/prepare-network.yml --list-hosts  # кого затронет
```

Kubespray — из клона, .venv

LOCAL 03_1.2bis_kubespray_anatomy/kubespray · KUBESPRAY_DIR="$PWD"

```
git clone --branch v2.31.0 --depth 1 \
  https://github.com/kubernetes-sigs/kubespray.git  # закреплённый release
git describe --tags --exact-match          # v2.31.0 — внутри клона
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
"$KUBESPRAY_DIR/.venv/bin/ansible" all -m ping  # SSH + Python, не ICMP
"$KUBESPRAY_DIR/.venv/bin/ansible" all --become -a 'whoami'  # sudo → root
"$KUBESPRAY_DIR/.venv/bin/ansible-playbook" playbooks/preflight.yml
.venv/bin/ansible-inventory -i inventory/video2/inventory.ini --graph
.venv/bin/ansible-playbook -i inventory/video2/inventory.ini \
  --become cluster.yml                      # установка, 30-60 мин
.venv/bin/ansible-playbook -i … --become upgrade-cluster.yml  # Блок 3
# scale.yml — добавление нод; reset.yml — снос. Сейчас не запускаем.
```

Версия: для v2.31.0 нужен ansible-core 2.18.x (`.venv/bin/ansible-playbook --version`). **PLAY RECAP:** у каждой ноды `failed=0` и `unreachable=0`; `changed` уже входит в `ok`; есть `ignored` / `rescued` — читай, что произошло.

yc — облако из терминала

LOCAL yc config list в кадр не выводить — там учётные данные

```
yc compute instance list                   # все VM: 5 своих + worker'ы
yc managed-kubernetes node-group list      # STATUS / SIZE группы нод
yc managed-kubernetes node-group list-nodes --name k8s-managed-workers
yc managed-kubernetes cluster get-credentials k8s-managed \
  --external --kubeconfig "$KUBECONFIG"    # kubeconfig с exec, без токена
yc quota-manager quota-limit get --quota-id compute.ssdDisks.size \
  --resource-type resource-manager.cloud --resource-id "$YC_CLOUD_ID"
yc quota-manager quota-request create --resource-type resource-manager.cloud \
  --resource-id "$YC_CLOUD_ID" \
  --desired-limit quota-id=compute.ssdDisks.size,value=<BYTES>
yc quota-manager quota-request list --resource-type resource-manager.cloud \
  --resource-id "$YC_CLOUD_ID"             # APPROVED / PENDING
```

kubectl с двумя контекстами и нода

Kubeconfig и контексты

LOCAL корень клона Kubespray · один kubectl, два контекста

```
kubectl --kubeconfig inventory/video2/artifacts/admin.conf config get-contexts
kubectl --kubeconfig inventory/video2/artifacts/admin.conf \
  config rename-context kubernetes-admin-cluster.local@cluster.local kubespray
export KUBECONFIG="${PWD}/inventory/video2/artifacts/recording.conf" # 0600
kubectl config rename-context "$(kubectl config current-context)" k8s-managed
kubectl config get-contexts                 # kubespray · k8s-managed, *=тек.
kubectl config use-context kubespray        # умолчание — свой кластер
kubectl config view                         # users: exec (managed) / cert
kubectl --context k8s-managed auth whoami   # IAM-аккаунт vs kubernetes-admin
```

Кластер и приложение

CLUSTER --context kubespray · то же с --context k8s-managed · из 08_1.6_verify_cluster

```
kubectl --context kubespray get nodes       # node1/2 control-plane, node3
kubectl --context kubespray -n kube-system get pods -o wide  # static pod'ы
kubectl --context kubespray apply -k app/   # ns + deploy + svc + pdb + client
kubectl --context kubespray -n traffic-lab \
  rollout status deployment/devops-may-cry --timeout=300s
kubectl --context kubespray -n traffic-lab get pods -o wide
kubectl --context kubespray -n traffic-lab get endpointslices \
  -l kubernetes.io/service-name=devops-may-cry -o yaml  # кто за Service
kubectl --context kubespray -n traffic-lab exec client -- \
  curl -fsS -w '\nHTTP %{http_code}\n' http://devops-may-cry/readyz
kubectl --context k8s-managed -n traffic-lab describe svc dmc-lb  # Events
kubectl --context kubespray version         # Server Version = kube_version
```

На ноде — systemd-сервисы через Ansible

NODE по SSH на все ноды группы: etcd и kubelet — systemd

```
.venv/bin/ansible -i inventory/video2/inventory.ini \
  etcd --become -m command -a 'systemctl is-active etcd'    # active ×3
.venv/bin/ansible -i inventory/video2/inventory.ini \
  k8s_cluster --become -m command -a 'systemctl is-active kubelet'
```

Глубже на ноде (в этом видео не запускали, приёмы из Блока 1): `systemctl is-active containerd`, `crictl ps`, `journalctl -u kubelet`. Static pod'ы control plane лежат в `/etc/kubernetes/manifests`.

AI КАК ТРЕНАЖЁР

Спроси нейронку правильно — по темам блока

AI не заменяет стенд — он усиливает. Правило то же, что в Блоке 1: проси **объяснить → дать команды для проверки → задать тебе вопрос**. Каждый ответ сверяй на своём стенде: у тебя есть state, playbook и два контекста kubectl.

ПОСЛЕ TERRAFORM

```
У меня Terraform-стенд: сеть + пять VM через модуль с for_each,
state локальный. Разведи без «магии» три вещи: конфигурация, state,
реальный объект. Объясни, чем drift отличается от обычного diff после
правки кода. Дай команды, которыми я найду drift у себя, и спроси,
что делать, если ручная правка была чьим-то хотфиксом.
```

↳ **проверь:** terraform plan -var-file=env/video.tfvars → No changes · terraform state list

ПОСЛЕ ANSIBLE

```
Я прогнал playbook дважды, второй раз changed=0. Объясни, за счёт
чего template / file / apt идемпотентны, где эта гарантия кончается
(shell с >>, service state: restarted) и почему handler сработал
на restore, но молчит на повторе. Задай мне 3 вопроса как на собесе.
```

↳ **проверь:** ansible-playbook playbooks/site.yml –check –diff → changed=0, RUNNING HANDLER нет

ПОСЛЕ KUBESPRAY

```
Я поставил кластер Kubespray v2.31.0 на 3 ноды: containerd, Calico,
kube-proxy iptables, etcd на всех трёх. Пройди 7 этапов cluster.yml
и скажи, что появляется на ноде после каждого: где etcd (systemd),
где api-server (static pod), почему ноды NotReady до сети Pod.
Дай команду проверить каждый этап на моих нодах.
```

↳ **проверь:** ansible etcd –become -m command -a 'systemctl is-active etcd' · kubectl -n kube-system get pods -o wide

ПОСЛЕ MANAGED VS SELF-MANAGED

```
У меня два контекста: kubespray и k8s-managed. Объясни границу
ответственности через то, что видно в командах: get nodes,
kube-system, auth whoami, Service LoadBalancer с <pending>.
Что в managed всё равно остаётся моим? Задай вопрос,
как отозвать доступ администратора в каждом из кластеров.
```

↳ **проверь:** kubectl –context k8s-managed auth whoami · kubectl config view (exec vs client-certificate-data)

ФИНАЛ · ИТОГ БЛОКА

Итог Блока 2 + что дальше в серии

Кластер больше не «появляется». Ты видел, откуда он берётся: три инструмента по слоям — **Terraform** создаёт машины, **Ansible** настраивает Linux, **Kubespray** собирает Kubernetes, — и рядом managed-кластер тем же языком. Один kubectl, два контекста, одно приложение в обоих.

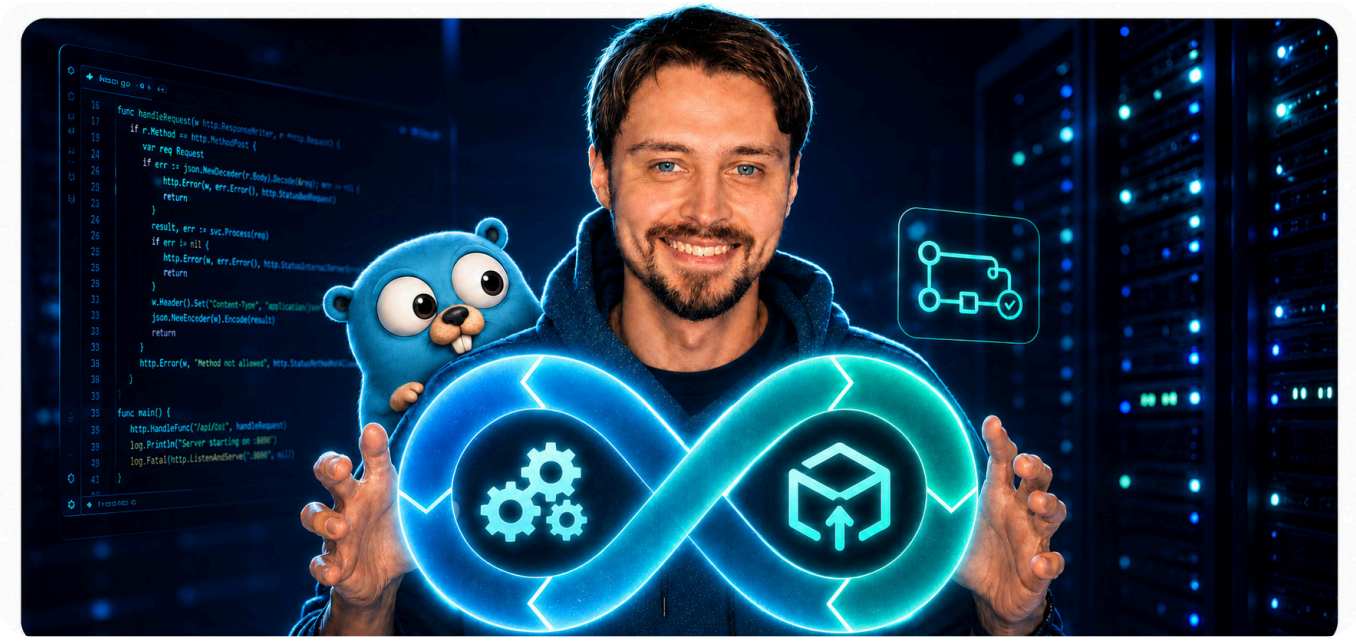
Что ты сделал руками

- **DEVOPS MAY CRY локально** — compose, `/livez` · `/readyz` · `POST /orders`, заявка в Postgres.
- **Hello Terraform** — init → validate → plan → apply, три доказательства, drift руками и возврат через apply.
- **Hello Ansible** — file walk, первый прогон, честный `changed=0`, precedence, drift с `--check --diff` и handler.
- **Пять VM Terraform-ом** — сеть, security group на `<YOUR_IP>/32`, node1–node3 + lb1, lb2 из одной карты машин.
- **Inventory из outputs** — `terraform output -raw` → `inventory.ini` и `all.yml`, ни одного адреса руками.
- **Preflight** — `-m ping`, sudo, ОС / Python / swap / overlay / br_netfilter / ip_forward; красный assert = стоп, не «повтори».
- **Kubespray cluster.yml** — 7 этапов, PLAY RECAP, `admin.conf`, три ноды Ready, etcd и kubelet как systemd.
- **Managed-кластер Terraform-ом** — сервисные аккаунты, control plane за API, квота SSD и Provisioning, kubeconfig с exec.
- **Приложение в обоих кластерах** — `apply -k`, rollout, EndpointSlice, curl из client pod; таблица границы ответственности.

Что дальше в серии — Блок 3

ТЕМА	ЧТО ЗАКРОЕМ	ЗАЧЕМ
MetalLB	Service type LoadBalancer в своём кластере получает адрес	сегодня `dmc-lb` у kubespray висит в `<pending>`
Внешний HA-вход	lb1 и lb2 из того же Terraform: VIP, два прокси, отказ одной машины	inventory для них уже лежит в `13_1.11_external_ha`
Путь трафика	от внешнего IP до пода по прыжкам: вход → NodePort → kube-proxy → Pod	объяснить на собесе, а не «оно работает»
Безопасное обновление	`upgrade-cluster.yml`, снимок etcd до, проверка приложения после	версия — твоя зона в self-managed

СОБЕС «Про этот проект за минуту: Terraform-ом поднял сеть и пять VM в облаке, адреса из outputs ушли в inventory без ручной правки. Ansible-ом проверил ноды preflight-ом, Kubespray v2.31.0 собрал кластер: containerd, Calico, etcd на трёх нодах, control plane на двух. Рядом тем же Terraform-ом — managed-кластер. Одни манифесты приложения работают в обоих; разница видна в командах: в managed нет control-plane нод, доступ через IAM, LoadBalancer получает адрес от облака. В своём — всё это моя зона, и я знаю, где именно».



За этой тетрадью стоит BoostMentor

Я Виктор — DevOps-инженер и ментор. Если хочешь продолжить со мной, записывайся на менторство: разберём твои вопросы, практические задачи и подготовку к собеседованиям. Подробности — на boostmentor.ru. Новые разборы и материалы публикую в Telegram и на YouTube.

Telegram-канал — t.me/booostmentor
разборы и следующие блоки серии. Забирай и повторяй руками

Исходники второй части — релиз video2b-v1.0 на GitHub
DEVOPS MAY CRY, Terraform, Ansible, настройки Kubespray и манифесты. На странице релиза выбери Source code (zip); карта файлов выпуска находится в VIDEO2B.md

Первая часть — Terraform, Ansible и создание VM
если хочешь посмотреть подготовку стенда; короткий итог уже есть в начале тетради

Записаться на менторство — boostmentor.ru
практика DevOps, разбор твоих вопросов и подготовка к собеседованиям